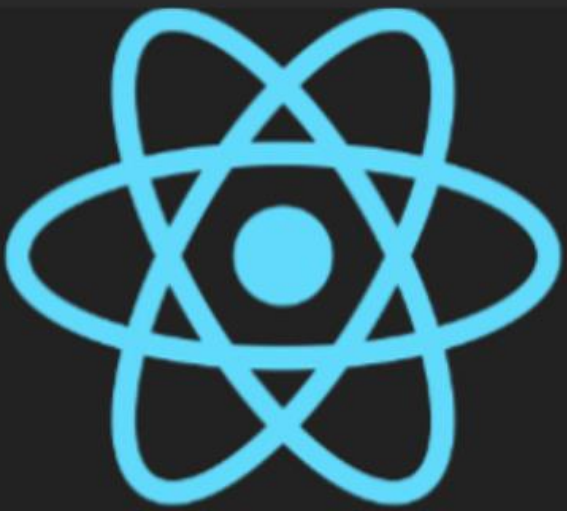
CRACKING THE REACTJS INTERVIEWS



BOOK BY RITESH, SUMIT & VAIBHAV

Contents

INTRODUCTION 3

Chapter 1: React.js Fundamentals 4

Chapter 2: Mastering JavaScript 95

Chapter 3: Frontend Concepts 161

NOTE:

This ebook has all Important and most asked React JS interview questions .

To dig deep intp React Js – please read Vol 1 and 2.

INTRODUCTION

Welcome to **Cracking the React Interview**. This book is crafted for developers who are looking to take their React skills to the next level and ace advanced interviews at top tech companies. React.js has become the go-to library for building dynamic, high-performance user interfaces, and mastering its core concepts is crucial for anyone serious about a career in frontend development. Whether you're preparing for interviews at leading firms or simply looking to deepen your knowledge of React, this book provides a comprehensive guide to tackle complex React topics with confidence.

In the fast-paced world of software development, React has evolved into a highly sophisticated and feature-rich library. Advanced React concepts such as hooks, context API, higher-order components, performance optimization, and state management are essential for building scalable and efficient applications. This book focuses on those concepts and presents them in the context of real-world scenarios and interview questions that test your practical knowledge and problem-solving abilities.

Chapter 1: React.js Fundamentals

1. What is Virtual DOM? How does the Virtual DOM work? What makes React fast?

Answer:

The Virtual DOM (VDOM) is an **in-memory representation of the actual DOM elements**. React creates a **virtual copy of the DOM** and keeps it in memory, allowing it to be more efficient when rendering updates to the UI.

Here's how the Virtual DOM works

1. Initial Render: When a React component is rendered for the first time, React creates a Virtual DOM tree (a lightweight copy of the real DOM).

2. State Changes: When the state or props of a component change, React creates a new Virtual DOM representing the UI with the updated state.

3. Diffing: React compares (or "diffs") the new Virtual DOM with the previous one using a highly optimized algorithm. This process is called reconciliation.

4. Updating the Real DOM: After detecting changes, React efficiently updates only the changed parts in the real DOM. It avoids re-rendering the entire DOM, thus minimizing the number of expensive DOM manipulations.

This approach makes React fast because:
Batching Updates: React batches multiple updates together to

reduce the number of real DOM updates.

Optimized Rendering: By re-rendering only the parts that change, React reduces the workload on the browser's rendering engine, leading to improved performance.

---

2. What is an ErrorBoundary in React?

Answer:

An ErrorBoundary is a React component that catches JavaScript errors anywhere in its child component tree and **displays a fallback UI when an error occurs**. Error boundaries catch errors during rendering, in lifecycle methods, and in constructors of the whole tree below them.

Error boundaries help in cases where something unexpected breaks the React application, and they can display user-friendly fallback content like an error message.

Key methods for implementing an ErrorBoundary:

getDerivedStateFromError(error): Used to render a fallback UI after an error is thrown

componentDidCatch(error, info): Used to log error information or perform side-effects like reporting errors to a monitoring service

Example:

```
import React from "react";

class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError = () => ({ hasError: true });
  render() {
    return this.state.hasError ? <h1>Error!</h1> : this.props.children;
  }
}

function BuggyComponent() {
  const [count, setCount] = React.useState(0);
  if (count > 2) throw new Error("Too high!");
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}

function App() {
  return (
    <ErrorBoundary>
      <BuggyComponent />
    </ErrorBoundary>
  );
}
```

---

3. How to implement code-splitting in React? Explain React.lazy and Suspense.

Answer:

Code-splitting is a technique to load only the parts of the application that are needed, rather than loading the entire app on the first load. This is crucial for improving the performance of large applications. React has built-in support for code-splitting using React.lazy() and Suspense.

React.lazy() allows you to load components dynamically by lazily loading them when they are needed. This helps in splitting the code and improving performance by reducing the initial bundle size

const LazyComponent = React.lazy(() => import('./LazyComponent'));

Suspense is a React component that is used to display a fallback UI (such as a loading spinner) while the lazily loaded component is being loaded

Example:

```
import React, { Suspense } from "react";

// Lazy-load a component
const LazyComponent = React.lazy(() => import("./LazyComponent"));

function App() {
  return (
    <div>
      <h1>Code-Splitting Demo</h1>
      {/* Suspense with fallback while LazyComponent loads */}
      <Suspense fallback={<div>Loading...</div>}>
        <LazyComponent />
      </Suspense>
    </div>
  );
}
```

In this example:

When LazyComponent is being loaded, React will display "Loading..." in the UI until the component is ready.

---

4. What are the rules of hooks?

Answer:

Hooks are a fundamental part of React for managing state and side effects in functional components. To ensure they work

correctly, React enforces the following rules

1. **Only call hooks at the top level:** Hooks should only be called at the top level of a functional component or custom hook. They should never be called inside loops, conditions, or nested functions. This is important because React needs to maintain the correct order of hooks on each render

❌ Incorrect:

```
if (condition) {
  useEffect(() => { ... }); // This is incorrect
}
```

✅ Correct:

```
useEffect(() => {
  if (condition) {
    // Perform side-effect
  }
}, [condition]);
```

2. **Only call hooks from React functions**: Hooks should only be called from functional components or custom hooks. You can't use hooks inside regular JavaScript functions.

❌ Incorrect:

```
function nonReactFunction() {
  const state = useState(); // Invalid usage
}
```

✅ Correct:

```
function MyComponent() {
  const state = useState(); // Valid usage
}
```

---

5. What does the useState hook return and

accept?

Answer:

The useState hook is used to declare state variables in functional components. It accepts the initial state and returns a pair of value

1. The current state value
2. A function to update the state

Example:

```
const [count, setCount] = useState(0);
```

count: The current state, initialized with 0.
setCount: A function that updates the state. When called, React re-renders the component with the updated state.

You can update the state directly, or you can use a function inside setCount to access the previous state:

```
setCount(prevCount => prevCount + 1);
```

This ensures that updates are made based on the previous state value, which is especially important in asynchronous situations like event handlers or side effects.

---

6. How to Update Object Properties Stored in the State?

Answer:

In React, when updating an object in the state, it's important to remember that state should be treated as immutable. This means you should not directly modify the object; instead, create a new copy of the object, update the relevant property, and then

set the new state.

Example:

```
import { useState } from 'react';
import { View, Button, Text } from 'react-native';

function MyComponent() {
  const [user, setUser] = useState({ name: 'ReactJS', age: 30 });

  const updateAge = () => {
    setUser({ ...user, age: 31 });
  };

  return (
    <View>
      <Text>{user.name}, {user.age}</Text>
      <Button title="Update Age" onPress={updateAge} />
    </View>
  );
}
```

In the above code:

The setUser function is used to update the state.

The spread operator (...prevUser) creates a shallow copy of the previous user object.
The age property is updated to 31, while other properties (like name) remain unchanged.

Another Example:

```
import { useState } from 'react';
import { View, Button, Text } from 'react-native';

function MyComponent() {
  const [user, setUser] = useState({ name: 'ReactJS', age: 30 });

  const updateAge = () => {
    setUser({ ...user, age: 31 });
  };

  return (
    <View>
      <Text>{user.name}, {user.age}</Text>
      <Button title="Update Age" onPress={updateAge} />
    </View>
  );
}
```

---

7. When will the below commented lines run?

a. No Dependency Array:

useEffect(() => {

console.log("No dependency array"); //1

});

Answer:

This useEffect runs on every render of the component because no dependency array is provided. Without a dependency array, useEffect executes after every render cycle.

b. With State Dependency:

```
useEffect(() => {
  console.log("state updated"); //2
  return () => {
    console.log("clean up"); //3
  };
}, [someState]);
```

Answer:

console.log("state updated"); // 2 runs every time someState changes.

The cleanup function console.log("clean up"); // 3 runs right before the next effect is invoked or when the component unmounts

---

8. What is the Difference Between a Controlled and Uncontrolled Component in React?

Answer:

In React, forms can be either controlled or uncontrolled, depending on how their data is handled.

Controlled Components: In controlled components, form data is handled by React's state. The value of the form element is bound to a state variable, and any updates to the form are managed through React's state via onChange handlers.

```
function ControlledInput() {
  const [text, setText] = useState('');

  return (
    <View>
      <TextInput
        value={text}
        onChangeText={setText}
        style={{ borderWidth: 1 }}
      />
    </View>
  );
}
```

Uncontrolled Components: In uncontrolled components, the form data is managed by the DOM itself, rather than React. To retrieve the input value, you typically use a ref to access the DOM node directly.

```
function UncontrolledInput() {
  const inputRef = useRef(null);

  const logValue = () => {
    console.log(inputRef.current.value);
  };

  return (
    <View>
      <TextInput ref={inputRef} style={{ borderWidth: 1 }} />
      <Button title="Log Value" onPress={logValue} />
    </View>
  );
}
```

Key Differences:

Controlled Components rely on React state for their value, providing more control and allowing for validation, formatting, or conditional updates.

Uncontrolled Components manage their own state internally via the DOM, leading to simpler code when you don't need to control the form's behavior tightly.

---

9. What is the difference between useRef and forwardRef ?

Answer:

useRef creates a reference to a DOM element or a React component that persists across renders without triggering a re-render. It can be used to directly access a DOM node, track previous values, or maintain state that doesn't cause re-renders.

```
function MyComponent() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <View>
      <TextInput ref={inputRef} style={{ borderWidth: 1 }} />
      <Button title="Focus" onPress={focusInput} />
    </View>
  );
}
```

forwardRef is used to pass a ref from a parent component

down to a child component. It allows a parent component to directly reference a DOM element or another component in its child.

```
import { forwardRef } from 'react';
import { TextInput, View } from 'react-native';

// Child component
const MyInput = forwardRef((props, ref) => (
  <TextInput ref={ref} style={{ borderWidth: 1 }} {...props} />
));

// Parent component
function MyComponent() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <View>
      <MyInput ref={inputRef} />
      <Button title="Focus" onPress={focusInput} />
    </View>
  );
}
```

---

10. What is the difference between useEffect and useLayoutEffect ?

Answer:

<u>useEffect:</u>
Runs after the DOM has been painted. This is the most commonly used hook for running side-effects like data fetching, subscriptions, or updating the DOM. Since it runs after the paint, it does not block the browser from updating the screen.

```
useEffect(() => {
  document.title = "New Title";
}, []);
```

useLayoutEffect:

Runs synchronously after all DOM mutations but before the browser has a chance to paint. This makes it useful for measuring DOM elements or making immediate updates that will affect the layout. This can block the paint until the effect is complete, which may cause a delay in rendering.

```
useLayoutEffect(() => {
  const height = document.getElementById("myDiv").offsetHeight;
  console.log(height);
}, []);
```

Key Difference:

useEffect is non-blocking and runs after the render is committed to the screen.

useLayoutEffect is blocking and runs before the paint, making it ideal for situations where DOM measurements or layout adjustments are needed.

Aspect	`useEffect`	`useLayoutEffect`
Timing	After paint (async)	Before paint (sync)
Execution	Non-blocking	Blocking
Use Case	Async tasks (e.g., fetch)	Layout tasks (e.g., refs)
Performance	Lighter, may flicker	Heavier, prevents flicker

---

11. What is useImperativeHandle hook?

Answer:

useImperativeHandle is a React hook that customizes the instance value exposed to parent components when using ref with forwardRef.
Allows a child component to expose specific methods or values to a parent, rather than the full DOM/native element.
Works with forwardRef to define what the ref.current contains.

```
const CustomInput = forwardRef((props, ref) => {
  const inputRef = useRef(null);

  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(),
    clear: () => inputRef.current.clear(),
  }));

  return <TextInput ref={inputRef} style={{ borderWidth: 1 }} />;
});

// Parent component
function MyComponent() {
  const inputRef = useRef(null);

  const handleFocus = () => {
    inputRef.current.focus(); // Calls custom method
  };

  return (
    <View>
      <CustomInput ref={inputRef} />
      <Button title="Focus Input" onPress={handleFocus} />
    </View>
  );
}
```

---

12. What is the difference between useMemo , useCallback and React.memo ?

Answer:

useMemo is used to memoize the result of a function to prevent recalculations on every render. It recalculates the value only if the dependencies change. It's useful for optimizing expensive calculations.

Example:

```
const memoizedValue = useMemo(() => expensiveComputation(data), [data]);
```

useCallback memoizes a function reference. It returns a memoized version of a callback function that only changes if one of its dependencies changes. This is helpful for preventing unnecessary re-creation of functions when passing them down as props.

Example:

```
const handleClick = useCallback(() => {
  console.log("Clicked");
}, []);
```

React.memo is a higher-order component (HOC) that wraps a component and memoizes its output. This means it will only re-render if its props have changed, preventing unnecessary renders.

Example:

```
const MyComponent = React.memo(function MyComponent({ data }) {
  return <div>{data}</div>;
});
```

Aspect	`useMemo`	`useCallback`	`React.memo`
Purpose	Memoize values	Memoize functions	Memoize components
Returns	Computed value	Function instance	Memoized component
Use Case	Expensive calculations	Stable callbacks	Prevent re-renders
Dependencies	Array of deps	Array of deps	Props (shallow compare)

13. Which unit testing library is commonly used with React?

Answer:

The most commonly used unit testing libraries for React are:

1. Jest: A testing framework developed by Facebook, which works seamlessly with React. It supports snapshot testing, mocking, and asynchronous tests.

2. React Testing Library: A library focused on testing React components by interacting with them as a user would. It encourages best practices by testing the component's behavior, not the implementation.

3. Enzyme (deprecated): A utility for testing React components, allowing shallow rendering, testing lifecycle methods, and more. It has been largely replaced by React Testing Library due to React 18 compatibility concerns.

---

14. What kind of comparison is done in the useEffect dependency array and when can it have issues?

Answer:

React will compare each dependency with its previous value using the Object.is comparison.

Now what is Object.is()?
Object.is() is almost same as the === operator. The only difference between Object.is() and === is in their treatment of **signed zeros and NaN values**. The === operator (and

the == operator) treats the number values -0 and +0 as equal, but treats NaN as not equal to each other.

JavaScript provides 2 data types: **primitives and structurals.**

The primitives are **numbers, booleans, strings, symbols, and special values null and undefined.**

The structural are **Objects(new Object, Array, Map, WeakMap, Date, Set) and Functions** .

Primitives are compared by value and structural are compared by reference .

```
const a = 1;
const b = 1;
const arr1 = [1];
const arr2 = [1];
const person = { name: "Batman" };
const batman = person;
const person2 = { name: "Batman", address: { city: "Gotham" } };

// Log comparisons from the provided example
console.log("Comparisons:");
console.log(a === b); // true (primitive, compared by value)
console.log(arr1 === arr2); // false (structural, compared by reference)
console.log(Object.is(person, batman)); // true (same reference)
console.log(Object.is(person2.name, person.name)); // true (primitive strings)
```

---

15. What are Higher-Order Components (HOCs)?

Answer:

A Higher-Order Component (HOC) is a function that **takes a component and returns a new component**. HOCs allow you to reuse logic across multiple components without modifying their implementation. They are similar to decorators.

<u>Example of an HOC:</u>

```
function withLogging(WrappedComponent) {
  return function(props) {
    useEffect(() => {
      console.log("Component mounted");
      return () => console.log("Component unmounted");
    }, []);
    return <WrappedComponent {...props} />;
  };
}
```

You can wrap any component with withLogging to log messages whenever it mounts or unmounts.

```
const EnhancedComponent = withLogging(MyComponent);
```

---

16. What is useContext/ Context API and why do we need it?

Answer:

The Context API in React allows you to create global data that can be accessed by any component in your component tree without needing to pass props manually at every level. It is useful for sharing data like themes, user authentication, or settings across deeply nested components.

useContext is a hook that allows you to access the value of a context in functional components.

Why do we need it? Without the Context API, **prop drilling** (passing props through multiple layers of components) becomes cumbersome. The Context API helps to avoid this by **providing a global store that can be accessed anywhere**.

<u>Example:</u>

```
const ThemeContext = React.createContext('light')
function App() {
```

```
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <div>{theme}</div>; // Outputs "dark"
}
```

---

17. What is Lifting State Up in React?

Answer:

Lifting state up refers to the practice of moving state from child components to a common parent component to enable shared state management.
This is necessary when multiple child components need to access or modify the same state.
By lifting the state up, you can ensure that all dependent components re-render with the updated state.

For instance, if two sibling components need to interact based on a shared input value, you would lift the input state to their parent component, allowing the parent to manage and pass down the value and the update function as props.

```react
import { useState } from 'react';
import { View, Text, Button } from 'react-native';

// Child Component 1
function CounterDisplay({ count }) {
  return <Text>Count: {count}</Text>;
}

// Child Component 2
function CounterButton({ onIncrement }) {
  return <Button title="Increment" onPress={onIncrement} />;
}

// Parent Component (State Lifted Here)
function MyComponent() {
  const [count, setCount] = useState(0);

  const handleIncrement = () => setCount(count + 1);

  return (
    <View>
      <CounterDisplay count={count} />
      <CounterButton onIncrement={handleIncrement} />
    </View>
  );
}
```

<u>Before Lifting (Problem)</u>
If count lived in CounterDisplay, CounterButton couldn't access or update it without prop drilling or duplication.

<u>After Lifting (Solution)</u>
count is lifted to MyComponent.
count passed to CounterDisplay, handleIncrement to CounterButton.
Both children sync via the parent's state.

---

18. What are the various methods of adding CSS

to a React application, and what are the benefits of each?

Answer:

In React, there are several ways to style components, each with its own advantages and use cases. Below are the most common methods, with explanations of when to choose one approach over the other:

1. CSS Stylesheets

The traditional way of styling is to use external or global CSS stylesheets. In this approach, you create a .css file and import it into your React component or project.

```
import './App.css';

function App() {
 return <div className="container">Hello World</div>;
}
```

When to Use:
When your project doesn't require component-specific or dynamic styles, using global CSS is the simplest solution.
When you want styles to be globally available across multiple components.

Drawbacks:
All classes are in the global scope, which can cause style collisions between components.
Specificity Issues: Managing CSS specificity can be challenging as your styles grow in complexity.

2. Inline Styles

Inline styles in React are written directly within the component as a JavaScript object, using camelCase for property names.

```
function App() {
  return <div style={{ backgroundColor: 'blue', color: 'white' }}>Hello
World</div>;
}
```

When to Use:
When you need to apply styles conditionally or dynamically based on component state or props.
Performance: Inline styles are great for small, specific, or highly dynamic styles (like setting widths, colors, etc., based on the component's state).
Scoped Styles: Inline styles are scoped to the element, so there's no risk of style leakage across components.

Drawbacks:
Lack of Pseudo-Classes: You can't use CSS features like :hover, :active, or media queries in inline styles.

No Reusability: Inline styles can't be reused across components, which can lead to duplication.

Verbose: When styling complex components, inline styles can become verbose and hard to manage.

3. CSS Modules

CSS Modules allow you to create CSS files where class and animation names are scoped locally to the component. This avoids conflicts in naming, ensuring that styles are unique to the component.

```
import styles from './App.module.css';

function App() {
return <div className={styles.container}>Hello World</div>;
}
```

When to Use:
Scoped Styles: When you want styles that are component-specific and want to avoid global CSS issues like class name

conflicts.
Reusability: You can still keep styles in separate files, but the scoping ensures that styles do not leak into other components.
Larger Applications: This approach is especially useful in large applications where keeping styles modular and isolated becomes critical.

Drawbacks:
Setup: Requires build setup (like webpack) to handle .module.css files.

Not Dynamic: CSS Modules don't easily handle dynamic or conditional styling. You may still need inline styles for that purpose.

4. Styled-Components (CSS-in-JS)

Styled-components is a popular CSS-in-JS library that allows you to write actual CSS code inside your JavaScript files. It generates unique class names for your styles, which ensures that styles are scoped to the component.

```
import styled from 'styled-components';

const Container = styled.div`
  background-color: blue;
  color: white;
`;

function App() {
  return <Container>Hello World</Container>;
}
```

When to Use:
Dynamic Styles: You can easily handle dynamic styling based on props or state. For example, you can change styles based on props passed to the component.
Styled-components are great when you want to keep styles tightly coupled with components in a modular fashion.
Reusability and Theming: You can easily reuse styled components across the app and integrate them with a theme

provider for consistent theming.
No Class Name Collisions: Since styled-components generate unique class names, you don't have to worry about class name collisions.

Drawbacks:
Learning Curve: If your team is more familiar with traditional CSS, there may be a learning curve with styled-components.
Performance: For very large applications, the additional JavaScript overhead of CSS-in-JS might affect performance.
SSR (Server-Side Rendering) Setup: If you need server-side rendering, additional setup is required to make styled-components work efficiently with SSR.

5. SASS/SCSS

SASS is a CSS preprocessor that allows you to use variables, nested rules, mixins, and more to write more maintainable CSS. You can use SASS in React by integrating it with a build tool like webpack.

```
// App.scss
$primary-color: blue;

.container {
  background-color: $primary-color;
  color: white;
}

import './App.scss';

function App() {
  return <div className="container">Hello World</div>;}
```

When to Use:
Complex Styling: When your styling needs to be more complex and modular, such as using variables, nesting, or mixins.
Maintainability: SASS is a great choice when you have a large CSS codebase and want to organize it in a more maintainable way.
Reusability: Mixins and variables allow for reusable styles and can make your CSS more DRY (Don't Repeat Yourself).

Drawbacks:
Global Scope: Like standard CSS, SASS styles can still be globally scoped unless you specifically modularize them with CSS Modules.
Setup Required: You need a build tool like webpack to preprocess the SASS files.

Choosing the Right Approach:
For simple, global styling: Traditional CSS Stylesheets or SASS work well when styles are shared across multiple components and aren't tied to dynamic changes.

For scoped, modular styling: Use CSS Modules or styled-components to avoid class name conflicts and ensure component-specific styling.

For dynamic styling: Inline styles or styled-components are ideal when styles need to change based on component state or props.

For complex applications with themes: Styled-components is excellent for handling themes and creating reusable, dynamic styles.

For teams with traditional CSS knowledge: Start with CSS Modules for scoping, and progressively introduce styled-components or CSS-in-JS solutions as the application grows.

---

19. Can you explain what Hot Module Replacement (HMR) is and how it benefits the development process?

Answer:

Hot Module Replacement (HMR) is a feature that allows modules in a JavaScript application to be replaced or updated while the application is running, without requiring a full page reload. This enhances the development experience by allowing developers to see changes immediately without losing the current application state.

HMR can significantly speed up the development process, especially in large applications, by reducing the feedback loop between code changes and browser updates.

---

20. What is the use of Parcel, Vite, Webpack?

Answer:

Webpack: A highly configurable and widely-used module bundler that processes and transforms JavaScript, CSS, images, and more. It has a powerful plugin system and supports features like code splitting and lazy loading. However, it can have a steeper learning curve due to its configuration complexity.

Parcel: A zero-config bundler that is easy to set up and use, with out-of-the-box support for various file types. It is designed for speed and offers features like automatic code splitting and hot module replacement. Parcel simplifies the bundling process for developers who want to focus on building rather than configuring.

Vite: A modern build tool that focuses on speed and developer experience. It uses native ES modules during development for faster hot module replacement and compiles the code with Rollup for production. Vite provides an optimized development server and is particularly suitable for modern frameworks like React and Vue.

Tool	Primary Use	Key Features	Best For
Parcel	A zero-configuration build tool that bundles web applications quickly and easily.	Automatic code splitting, hot module replacement (HMR), built-in asset handling (e.g., CSS, images).	Small to medium projects, rapid prototyping.
Vite	A fast build tool and development server leveraging native ES modules for quick startup times.	Lightning-fast HMR, native ES module support, optimized production builds with Rollup.	Modern frontend projects, Vue/React apps.
Webpack	A highly configurable module bundler for complex JavaScript applications.	Extensive plugin ecosystem, tree shaking, code splitting, fine-grained control over builds.	Large, complex projects needing customization.

---

21. What is Tree Shaking?

Answer:

Tree shaking is a dead code elimination technique used in modern JavaScript bundlers (like Webpack and Rollup) to reduce the size of the final bundle by excluding unused code.

It works by analyzing the module dependencies and only including the code that is actually used in the application.

This is important for optimizing performance, reducing load times, and improving the user experience by minimizing the amount of JavaScript that needs to be downloaded and executed.

Imagine you have a module like this:

```
// utils.js
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}
```

And in your main file, you only use one function:

```
// main.js
import { add } from './utils.js';

console.log(add(5, 3));
```

With tree shaking, the build tool (e.g., Webpack, Rollup, or Vite) will see that subtract is never imported or used. It will exclude subtract from the final bundle, reducing the file size and improving performance.

Key Requirements for Tree Shaking:

1. ES6 Modules: Tree shaking only works with import/export syntax, not dynamic imports or CommonJS (require).
2. Build Tool Support: Tools like Webpack, Rollup, or Parcel implement tree shaking. For example:
   - Webpack enables it in production mode with optimization settings.
   - Rollup is designed with tree shaking as a core feature.
3. Side-Effect-Free Code: If a module has side effects (e.g., modifying global variables), the tool might not remove it, even if it appears unused. You can mark modules as side-effect-free using a package.json field like "sideEffects": false.

---

22. What is the difference between console.log(<HeaderComponent/>) and console.log(HeaderComponent()) in React?

Answer:

console.log(<HeaderComponent/>):

This logs a React element (JSX). It represents the virtual DOM structure that React will render, including props and state. This is how React components are typically rendered in the UI.

console.log(HeaderComponent()):

This logs the result of invoking the HeaderComponent function. If HeaderComponent is a functional component, it will execute the component logic and return the rendered output, which may not include any React-specific behavior like hooks or lifecycle methods unless executed within a React environment.

---

23. What is React.Fragment?

Answer:

React.Fragment is a component that allows you to group multiple child components without adding extra nodes to the DOM. It helps to avoid unnecessary wrapper elements, which can improve styling and layout.

You can use React.Fragment in situations where you need to return multiple elements from a component but do not want to introduce additional markup. An alternative shorthand syntax is <>...</>.

---

24. If two components share the same state, how does changing the state in one component affect the other?

Answer:

If two components share the same state through a common parent (lifting state up), changing the state in one component will trigger a re-render of both components. This is because the state resides in the parent component, and any updates will propagate down to the child components, allowing them to reflect the new state. If the components maintain their own independent states, then changes in one component will not affect the other.

---

25. How does React manage routing and navigation, and what libraries can be used for this purpose?

Answer:

React manages routing and navigation through libraries such as React Router, which provides a declarative approach to defining routes and rendering components based on the current URL. With React Router, developers can define routes using the <Route> component, manage navigation using the <Link> component, and handle nested routes through nested <Route> configurations. It also allows for route parameters, redirects, and route guarding, enabling a complete navigation experience within a single-page application (SPA).

---

26. What performance optimization techniques can you implement in React applications?

Answer:

Some common performance optimization techniques in React applications include:

1. Memoization: Use React.memo for functional components and

PureComponent for class components to prevent unnecessary re-renders when props haven't changed.
2. Code Splitting: Implement code splitting to load only the necessary code, as discussed earlier.
3. Lazy Loading: Use React.lazy and Suspense to delay the loading of components until they are needed.
4. Avoiding Anonymous Functions: Define functions outside of render methods to prevent creating new instances on each render.
5. Use the React Profiler: Profile your application to identify performance bottlenecks and optimize accordingly.
6. Batching State Updates: Utilize batching to combine multiple state updates into a single re-render, reducing the number of renders triggered.

---

27. What is the significance of the key prop in React lists? What could go wrong if the keys are not unique?

Answer:

The key prop in React helps identify list elements uniquely, allowing React to efficiently update the UI.

Significance of key:

1. Efficient Updates: React uses key to match elements between renders, updating only what has changed.
2. Prevents Unnecessary Re-renders: Unique keys help React avoid re-rendering unchanged elements.
3. State Preservation: Correct keys allow React to maintain component state across renders.

Issues with Non-unique Keys:
1. Incorrect Updates: React may confuse elements, leading to incorrect updates.

2. Lost State: Components might lose their internal state if React can't correctly identify them.

---

28. What will happen when you start a timer inside useeffect but move to the next screen before the timer ends?

Answer:

When you start a timer inside useEffect (e.g., using setTimeout or setInterval) but navigate to another screen before the timer ends, **the timer will still continue running unless you explicitly clear it**. This can lead to memory leaks or unintended side effects if the component is unmounted and the timer tries to execute code on an unmounted component.

When you move to the next screen in a React app (especially in cases like React Router or React Navigation), the previous component is often unmounted.
If you don't clear the timer, it will attempt to execute the callback, potentially trying to update the state of an unmounted component, which can cause issues such as memory leaks.

<u>Example Scenario:</u>

```
useEffect(() => {
  const timer = setTimeout(() => {
    console.log('Timer finished');
    // Some state update or operation
  }, 5000);

  // Clean up the timer on component unmount
  return () => clearTimeout(timer);
}, []);
```

<u>In this case:</u>

If you navigate away (the component is unmounted) before the 5 seconds timer ends, the cleanup function (the return function of useEffect) will be executed.
The clearTimeout(timer) will prevent the callback from running and avoid potential memory leaks or unwanted operations on an unmounted component.

Best Practice:

Always clean up any asynchronous operations (timers, subscriptions, etc.) inside useEffect to avoid issues when the component is unmounted:

```
useEffect(() => {
  const timer = setTimeout(() => {
    // Logic to be executed
  }, 5000);

  return () => {
    clearTimeout(timer); // Cleanup on unmount
  };
}, []);
```

This ensures the timer is cleared when the component unmounts, preventing any side effects or memory leaks.

---

29. What is Jest, and why is it used in React applications?

Answer:

Jest is a **JavaScript testing framework maintained by Facebook**, widely used for testing React applications. It provides built-in test runners, assertions, mocking capabilities, and tools for isolating individual tests.

Jest simplifies writing unit, integration, and snapshot tests by providing an all-in-one solution. It's particularly popular in

React ecosystems because it’s optimized for working with React components.

Jest also includes features like parallel test execution, easy mocking, and excellent integration with tools like Babel and Webpack. It has a powerful runner for managing test cases and can run tests in parallel, which improves speed in large applications.

---

30. How do you mock modules in Jest?

Answer:

Jest allows you to mock modules by using jest.mock(). This function replaces the module with a mocked version during the test run. For instance, if you want to mock a third-party library or internal module:

```
jest.mock('axios', () => ({
  get: jest.fn(() => Promise.resolve({ data: {} })),
}));
```

In this example, the axios.get method is replaced with a mock function that always returns a resolved promise. Jest allows you to control the module's behavior, simulating different test scenarios like errors or asynchronous responses. You can also mock specific parts of a module using jest.spyOn(), which allows you to mock specific functions while keeping the rest of the module intact.

---

31. What is snapshot testing in Jest, and how do you use it?

Answer:

Snapshot testing is used to capture a "snapshot" of a rendered component's output (HTML structure). This snapshot is saved to a file during the first test run and used in subsequent runs to detect changes. For example:

```
import { render } from '@testing-library/react';
import MyComponent from './MyComponent';

test('renders correctly', () => {
  const { container } = render(<MyComponent />);
  expect(container).toMatchSnapshot();
});
```

This generates a snapshot of the rendered DOM structure in a __snapshots__ folder. If the component's output changes, the test will fail, prompting a review. Snapshot tests are beneficial for ensuring UI consistency but should be used carefully as they can break with minor changes. If a change is intentional, you can update the snapshot using jest --updateSnapshot.

---

32. How do you test asynchronous code with Jest?

Answer:

Asynchronous code can be tested using either promises or async/await in Jest. Jest waits for promises to resolve, allowing tests to pass or fail based on asynchronous operations. For example:

```
test('fetches data successfully', async () => {
  const data = await fetchData();
  expect(data).toEqual({ success: true });
});
```

For promise-based functions, you can test like this:

```
test('fetches data with promises', () => {
  return fetchData().then((data) => {
    expect(data).toEqual({ success: true });
  });
```

```
});
```

Jest also provides .resolves and .rejects matchers to test promise outcomes:

```
expect(fetchData()).resolves.toEqual({ success: true });
expect(fetchData()).rejects.toThrow('Error message');
```

---

33. What is React Testing Library, and how does it differ from Enzyme?

Answer:

React Testing Library (RTL) is focused **on testing user behavior and interactions rather than implementation details**.

The core philosophy of RTL is to **test the app from the user's perspective**. Instead of accessing internal component details like state, props, or lifecycle methods (which Enzyme allows), RTL encourages testing the rendered output and how the component behaves in response to user actions or state changes. This aligns with best practices for maintainable tests, focusing on the "what" rather than the "how" of a component's behavior. This makes tests more robust and less prone to breaking due to internal refactors.

---

34. How do you select elements in React Testing Library?

Answer:

React Testing Library provides a set of queries that follow a

user-centric approach. Some of the most common queries are:

getByRole: Selects elements by their accessibility role (e.g., button, link, heading).

getByText: Selects elements by their visible text content.

getByLabelText: Selects form inputs by the label text associated with them.

getByTestId: Selects elements using the data-testid attribute.

Example:

```
const { getByText, getByRole } = render(<MyComponent />);
expect(getByText('Submit')).toBeInTheDocument();
expect(getByRole('button')).toBeDisabled();
```

---

35. How do you test user interactions in React Testing Library?

Answer:

User interactions are simulated using fireEvent or userEvent. fireEvent provides lower-level interaction simulation (e.g., clicks, input changes), while userEvent simulates real-world interactions more closely. Example:

```
import { fireEvent, render } from '@testing-library/react';
import MyComponent from './MyComponent';

test('simulates user click', () => {
  const { getByRole } = render(<MyComponent />);
  const button = getByRole('button');
  fireEvent.click(button);
  expect(button).toBeDisabled();
});
```

userEvent simulates more realistic input behavior, like typing

into input fields:

```
import userEvent from '@testing-library/user-event';

test('types into input field', () => {
  const { getByPlaceholderText } = render(<MyComponent />);
  const input = getByPlaceholderText('Enter name');
  userEvent.type(input, 'John');
  expect(input.value).toBe('John');
});
```

---

36. What are the best practices for testing React components using React Testing Library?

Answer:

Focus on user interactions: Test components based on how users interact with them (e.g., clicks, form submissions), not on internal state or implementation details.

Use role-based queries: Prefer queries like getByRole or getByText over getByTestId, as they reflect how users would navigate the UI.

Avoid testing implementation details: Tests should be robust and independent of internal changes, such as refactoring component logic or updating class names.

Use waitFor for async tasks: Use waitFor for tasks that involve asynchronous behavior like data fetching or delayed UI updates. It ensures your test does not fail prematurely if the component hasn't finished rendering or updating.

Test accessibility: When testing components, ensure that they are accessible by using queries like getByRole and by checking ARIA attributes. Accessibility should be a key part of your testing strategy to ensure that the application works well with

screen readers and other assistive technologies.

Use userEvent for realistic user interactions: Whenever possible, prefer userEvent over fireEvent, as it more closely simulates how a real user would interact with the application (e.g., typing, clicking, or selecting options). This creates more accurate and reliable tests.

---

37. How can you test performance optimizations (like React.memo) in React components?

Answer:

To test if optimizations like React.memo are effective, you can use Jest's spyOn() method to monitor if the component re-renders unnecessarily. First, wrap the component with React.memo, and then spy on the component's render method:

```
const MemoizedComponent = React.memo(MyComponent);

test('does not re-render unnecessarily', () => {
  const spy = jest.spyOn(MemoizedComponent.prototype, 'render');

  const { rerender } = render(<MemoizedComponent propA="value1" />);
  rerender(<MemoizedComponent propA="value1" />); // Rerender with
same props

  expect(spy).toHaveBeenCalledTimes(1); // It should not re-render

  rerender(<MemoizedComponent propA="value2" />); // Rerender with
different props
  expect(spy).toHaveBeenCalledTimes(2); // It should re-render once
});
```

---

38. Explain how you would handle accessibility (a11y) in a large-scale React application.

Answer:

Cover below points:

Semantic HTML: Use proper HTML5 elements (e.g., <button>, <header>, <nav>) instead of div or span to ensure screen readers can interpret the content correctly.

Aria attributes: Use ARIA attributes (e.g., role, aria-label) to enhance the accessibility of components that do not have a semantic equivalent.

Keyboard navigation: Ensure all interactive elements are focusable and operable via the keyboard.

Accessible forms: Use proper labels with form elements and ensure all form controls are accessible to screen readers.

Testing: Regularly test with accessibility tools such as Lighthouse, axe, or screen readers to ensure the application meets accessibility standards.

Focus management: Handle focus correctly when navigating between components or after performing an action (e.g., closing a modal).

---

39. What are some best practices for creating custom hooks?

Answer:

Some best practices for creating custom hooks are:

Naming: Always start your hook name with “use” to follow React's conventions.

Separation of Concerns: Ensure the hook handles only one

specific task or logic.

Reusability: Aim to make your custom hooks reusable and not tightly coupled to a specific component.

Side effects: Handle side effects (e.g., subscriptions, cleanup) inside the custom hook using useEffect.

---

40. When Would You Prefer Custom Hooks Over Higher-Order Components (HOCs) or Render Props?

Answer:

Talk about below points:

1. Cleaner Code: Custom hooks keep your component structure simple and avoid the "wrapper hell" associated with HOCs and the verbosity of render props. They allow you to reuse logic without altering the component tree.

2. Readability: Hooks are functions that follow a straightforward pattern, making the code easier to read and maintain compared to nesting multiple HOCs or render props.

3. Performance: Custom hooks don't add extra layers to the component tree, which can happen with HOCs. This leads to more efficient and performant code.

4. Flexibility: Custom hooks allow you to reuse stateful logic in functional components, whereas HOCs and render props are more tied to component-based architectures.

Use custom hooks when you want to simplify and reuse logic without affecting your component structure.

---

41. Explain all new features in React 18.

Answer:

React 18, released in March 2022, introduced concurrency and performance improvements:

1. **Concurrent Rendering**:
A new rendering engine that allows React to pause, resume, or abandon renders for better performance and responsiveness.

2. Automatic Batching:
Groups multiple state updates into a single re-render, even outside event handlers (e.g., in promises or timeouts), reducing unnecessary renders.

3. Transitions (startTransition API):
Marks non-urgent updates (e.g., UI transitions) to prioritize urgent updates (e.g., user input), improving perceived performance.

4. Suspense for Server-Side Rendering (SSR):
Enhanced Suspense to support streaming SSR, allowing HTML to be sent incrementally with placeholders for slow components.

5. New Root API (createRoot):
Replaces ReactDOM.render with createRoot, enabling concurrent features and better hydration control.

6. Selective Hydration:
Hydrates components as their data resolves, rather than waiting for the entire tree, improving SSR interactivity.

7. useTransition Hook:
Provides a way to manage pending states during transitions, with an isPending flag.

8. useDeferredValue Hook:
Defers rendering of non-urgent UI parts until urgent updates are complete, similar to debouncing but interruptible.

9. useId Hook:
Generates unique IDs for accessibility purposes, stable across server and client renders.

10. useSyncExternalStore Hook:
Integrates external state (e.g., browser APIs) with React, recommended for library authors.

11. useInsertionEffect Hook:
Allows CSS-in-JS libraries to inject styles before layout effects, primarily for library authors.

---

42. Explain all new features in React 19.

Answer:

React 19, released on April 25, 2024, builds on React 18 with a focus on simplification, performance, and server-side capabilities:

1. **React Server Components (RSC):**
Components can render on the server at build time or per request, reducing client-side JavaScript and improving load times.

2. **Actions:**
Simplifies form submissions and data mutations with async functions, handling pending states, errors, and optimistic updates automatically.

3. **use Hook:**
A versatile hook to read promises, context, or other resources during render, replacing some uses of useEffect and useContext.

4. **React Compiler:**
Automatically optimizes code (e.g., memoization) without manual useMemo or useCallback, boosting performance.

5. **Document Metadata Support:**
Native support for <title>, <meta>, and <link> tags in components, hoisted to the document head, simplifying SEO and metadata management.

6. **Improved Suspense for SSR:**
Enhanced Suspense for server-side rendering, pausing rendering until data is ready, improving responsiveness.

7. **Asset Loading with Suspense:**
Background loading of images, scripts, and stylesheets with Suspense, reducing flicker and wait times.

8. **useActionState Hook:**
Manages state, pending status, and errors for Actions, streamlining form handling.

9. **useOptimistic Hook:**
Enables optimistic UI updates (e.g., showing changes before server confirmation) with automatic rollback on failure.

10. **useFormStatus Hook:**
Provides form submission status (e.g., pending) for better UX during Actions.

11. **Refs as Props:**
Refs can be passed as regular props without forwardRef, simplifying ref handling.

12. **Resource Loading APIs:**
New APIs like preload and preinit give control over when assets (e.g., scripts, styles) load or initialize.

13. **Directives (use client, use server):**
Marks code as client-only or server-only, integrating with Server Components (not a React 19 feature per se, but stabilized).

14. **Simplified Stylesheet Management:**
Better handling of <link rel="stylesheet"> and <style> tags, ensuring correct DOM positioning.

---

43. Can you name all React hooks that you know?

Answer:

React Hooks were first introduced in React 16.8 and additional hooks have been added in subsequent versions, such as React 18.

Below are all the hooks with version they were introduced in:

Hook Name	Version Introduced	Usage
`useState`	16.8	Adds state to functional components; returns a state variable and a function to update it.
`useEffect`	16.8	Handles side effects (e.g., data fetching, subscriptions) in functional components.
`useContext`	16.8	Accesses context values from a React context provider without prop drilling.
`useReducer`	16.8	Manages complex state logic with a reducer function, an alternative to `useState`.
`useCallback`	16.8	Memoizes callback functions to prevent unnecessary re-renders in child components.
`useMemo`	16.8	Memoizes expensive computations to optimize performance by caching results.
`useRef`	16.8	Creates a mutable reference that persists across renders (e.g., for DOM access).
`useImperativeHandle`	16.8	Customizes the instance value exposed to parent components when using `ref`.
`useLayoutEffect`	16.8	Similar to `useEffect`, but runs synchronously after DOM mutations, before painting.
`useDebugValue`	16.8	Displays a custom label in React DevTools for debugging custom hooks.
`useId`	18.0	Generates unique IDs stable across server and client for accessibility or form elements.
`useTransition`	18.0	Marks state updates as non-blocking, allowing UI to remain responsive during transitions.
`useDeferredValue`	18.0	Defers rendering of a value to improve performance for non-critical UI updates.
`useSyncExternalStore`	18.0	Subscribes to an external store and syncs it with React state (useful for libraries).
`useInsertionEffect`	18.0	Runs before DOM mutations for injecting styles (primarily for CSS-in-JS libraries).

`use`	19.0	Reads values from resources like Promises or context; can be used conditionally in render.
`useActionState`	19.0	Manages state for form actions, handling pending states and results of async submissions.
`useFormStatus`	19.0	Accesses the status (e.g., pending, method) of a parent form from a child component.
`useOptimistic`	19.0	Enables optimistic UI updates during async operations, reverting if the action fails.

---

44. What is the useId hook in React 18, and how does it solve issues related to accessibility or server-side rendering (SSR)?

Answer:

useId is a new hook introduced in React 18 that **helps generate unique IDs for accessibility and SSR**. In applications with SSR, IDs generated on the server and client can mismatch, leading to hydration issues. useId ensures that unique, deterministic IDs are generated consistently across the server and client, preventing issues when components rehydrate.

Example:

```
function Form() {
 const id = useId();
 return (
  <div>
   <label htmlFor={id}>Username</label>
   <input id={id} type="text" />
  </div>
 );
}
```

When to use:
To generate IDs for form elements like labels and inputs, especially in scenarios where SSR is used.

For accessibility, when each element needs a unique ID.

It ensures that the same ID is generated during both the server-side rendering phase and the client-side hydration phase, avoiding mismatches.

---

45. How does the useTransition hook work in React 18, and when would you use it to improve user experience?

Answer:

useTransition allows you **to mark certain updates as "non-urgent" or "low-priority."** This is particularly useful in situations where you want to keep the UI responsive while performing longer operations like data fetching or rendering complex components.

The hook returns a boolean (isPending) and a function (startTransition) to wrap non-urgent updates.

Example:

```
const [isPending, startTransition] = useTransition();
const [value, setValue] = useState("");

function handleInputChange(e) {
  startTransition(() => {
    setValue(e.target.value);
  });
}
```

When to use:
Debounced search results: You may want to show the user's input immediately but delay updating the list of results until after the user finishes typing.

Rendering large datasets: When rendering data that could block the UI, use useTransition to keep the interface responsive.

In a search input, the immediate input update is urgent, but fetching search results can be deferred to avoid freezing the UI. This helps with a smoother user experience.

---

46. Explain the concept of useDeferredValue in React 18. How does it differ from useTransition, and when should you use one over the other?

Answer:

useDeferredValue **delays updating a value until after more urgent rendering tasks are complete**. Unlike useTransition, which schedules a state update, useDeferredValue simply returns a deferred version of the value that takes longer to update.

Example:

```
const deferredValue = useDeferredValue(value);

return (
  <ExpensiveComponent value={deferredValue} />);
```

When to use:
Heavy computations: When you want to compute or render something that's based on a changing value, but you want to prevent that calculation from slowing down more important updates (like typing).

Rendering lists: For instance, if the user is typing a filter term, the UI should remain responsive while the actual filtered list updates more slowly.

Difference from useTransition:
useTransition wraps an update function and makes the entire

transition non-urgent, whereas useDeferredValue delays only the value and not the entire state update.

Use useDeferredValue for low-priority values that don't need to be rendered instantly but still depend on more immediate inputs.

---

47. What is the useInsertionEffect hook, and how does it differ from useLayoutEffect and useEffect?

Answer:

useInsertionEffect is **used for injecting styles during the "commit" phase, just before the browser paints the screen**.

This is particularly useful when working with CSS-in-JS libraries, **where you need to insert critical styles immediately before any DOM mutations happen**.

<u>When to use:</u>

For libraries like Emotion or styled-components, you might need to inject styles that affect the layout or visual appearance of components. **useInsertionEffect runs before useLayoutEffect** and DOM mutations, making it ideal for style injection.

<u>Difference:</u>

useEffect runs after the render and the paint.
useLayoutEffect runs after DOM mutations but before the paint.
useInsertionEffect runs during the render cycle, right before the DOM is mutated, making it critical for situations where styles must be injected in time for layout calculations.

---

48. How has useEffect behavior changed in React 18 concurrent rendering, and what considerations should you make when migrating to React 18?

Answer:

In React 18's concurrent rendering mode, useEffect might be delayed or interrupted due to concurrent rendering. This means that side effects might not happen immediately after a render.

Considerations:
Ensure that effects don't rely on synchronous state changes or assume that the component has rendered immediately.
Avoid side effects that affect UI elements directly in useEffect, as the effect may be delayed.

Example:
In non-concurrent mode, the useEffect hook typically runs after the DOM has been updated, but in concurrent mode, React may pause or restart the rendering process, so effects could be delayed or skipped during incomplete renders.

---

49. Can you explain how the useSyncExternalStore hook works in React 18? Why is it introduced, and how does it support external store subscriptions?

Answer:

useSyncExternalStore **provides a way for React components to subscribe to an external store** (e.g., Redux, Zustand) and ensure consistent state with concurrent rendering.

Example:

```
const state = useSyncExternalStore(store.subscribe, store.getState);
```

Why introduced:
Ensures that the component remains consistent with the external store's state, even when React is rendering concurrently or server-side rendering is involved.

Use case:
Useful for libraries like Redux, which handle global state outside of React's internal state system. It ensures that state snapshots are always up-to-date, even in concurrent rendering mode.

---

50. What is the purpose of the useImperativeHandle hook, and how is its behavior affected by React 18's concurrent rendering?

Answer:

useImperativeHandle allows you to customize the value that is exposed to parent components when using ref. It is mostly used in combination with forwardRef.

Example:

```
useImperativeHandle(ref, () => ({
  focus: () => {
    inputRef.current.focus();
  }
}));
```

In React 18, with concurrent rendering, the usage of useImperativeHandle must be done carefully since React might schedule updates differently. However, the actual imperative methods will still work predictably once the DOM update is committed.

---

51. What problem does useSyncExternalStore solve that wasn't adequately addressed by useEffect or other hooks in previous React versions?

Answer:

In React 17 and below, using useEffect for subscribing to external stores could lead to inconsistent states, especially during concurrent rendering or rehydration in SSR.

useSyncExternalStore ensures that the store's state is consistently synchronized with the component, avoiding race conditions and providing better support for concurrent rendering.

---

52. How does React 18's introduction of concurrent features affect hooks like useMemo and useCallback? Are there any changes in how these hooks work in concurrent rendering mode?

Answer:

In React 18, useMemo and useCallback still work the same way, but their recalculation might be deferred during concurrent rendering. However, React guarantees that once the component is committed, the memoized values will be consistent.

Key change: Memoization is still useful for optimizing performance, but developers need to ensure they don't rely on synchronous recalculations in concurrent rendering scenarios.

---

53. How can you combine useTransition and useDeferredValue to optimize both immediate and deferred updates in React 18?

Answer:

useTransition can handle deferring non-urgent state updates, while useDeferredValue can defer rendering based on low-priority values. You can combine these two to optimize UI responsiveness.

Example: Use useTransition to handle the overall state update, like when submitting a search form, and useDeferredValue to delay re-rendering a large list of search results based on the search term.

In scenarios like searching or filtering, you can use useTransition to ensure the input remains responsive, while useDeferredValue defers the rendering of large datasets or UI changes.

---

54. What is a React Portal, and when would you use it?

Answer:

A React Portal provides a way to render children into a DOM node that exists outside the DOM hierarchy of the parent component.

Portals are typically used to break out of the normal parent-child DOM structure.

Example:

```
ReactDOM.createPortal(child, container)
```

When to use:
Modals or Dialogs: Portals are perfect for rendering modals that need to appear above everything else but are not confined by their parent component's styles or positioning.

Tooltips: When you want to render tooltips outside the bounds of the parent, especially if the parent has overflow or clipping.

Context menus: Portals can ensure that context menus are rendered above other UI elements without being restricted by parent CSS.

---

55. How do you create a React Portal, and what is the significance of the second argument in ReactDOM.createPortal?

Answer:

To create a React Portal, you use the ReactDOM.createPortal method. The second argument in createPortal is the DOM element where the child components will be rendered.

Example:

```
return ReactDOM.createPortal(
  <div>Content outside parent hierarchy</div>,
  document.getElementById('modal-root')
);
```

The second argument (document.getElementById('modal-root')) is the target DOM node where the content will be rendered, allowing you to control where the component is injected in the DOM.

---

55. What are the common use cases where React Portals are preferred over standard component rendering?

Answer:

React Portals are preferred in the following scenarios:

1. Modals: For modals that need to be rendered outside of a component tree, especially to avoid issues with z-index or overflow.

2. Popups and Tooltips: When positioning relative to a parent element becomes difficult or when components are clipped due to parent overflow.

3. Notifications: Global notifications that should not be restricted by their parent's styles.

---

56. How would you update a deeply nested state value like location in the following state structure using useState?

```
const [state, setState] = useState({
  user: {
    profile: {
      details: {
        age: 30,
        location: 'Paris'
      }
    }
  }
});
```

Answer:

You can update the deeply nested location value like this:

```
setState(prevState => ({
  ...prevState,
  user: {
    ...prevState.user,
    profile: {
      ...prevState.user.profile,
      details: {
        ...prevState.user.profile.details,
        location: 'London' // Updated value
      }
    }
  }
}));
```

---

57. How would you modify the age field in the deeply nested state structure below?

```
const [state, setState] = useState({
  person: {
    info: {
      bio: {
        age: 25,
        city: 'New York'
      }
    }
  }
});
```

Answer:

To update the age field:

```
setState(prevState => ({
  ...prevState,
  person: {
    ...prevState.person,
    info: {
      ...prevState.person.info,
      bio: {
        ...prevState.person.info.bio,
        age: 26 // Updated value
      }
    }
  }
}));
```

---

58. You have a React component rendering a large list of 10,000 items, which causes performance issues when scrolling or re-rendering. How would you optimize rendering such a large list to improve the performance?

Answer:

The solution would be to implement windowing or virtualization using libraries like react-window or react-virtualized, which render only the visible portion of the list in the viewport.

<u>Example using react-window:</u>

```
import { FixedSizeList as List } from 'react-window';

const Row = ({ index, style }) => (
  <div style={style}>Item {index}</div>
);

const LargeList = () => (
  <List
    height={500}
    itemCount={10000}
    itemSize={35}
```

```
  width={300}
 >
  {Row}
 </List>
);
```

This renders only the visible items while scrolling, improving performance significantly.

---

59. You have a large form where each input field is a controlled component. The entire form re-renders whenever the state of any input changes, leading to performance bottlenecks. How would you prevent unnecessary re-renders while maintaining controlled inputs?

Answer:

You can optimize re-renders by splitting the form into smaller components and using React.memo() to memoize each field. Additionally, use useCallback() or useMemo() for functions passed as props to avoid unnecessary re-renders.

```
const InputField = React.memo(({ value, onChange, label }) => (
 <div>
  <label>{label}</label>
  <input value={value} onChange={onChange} />
 </div>
));

// Parent Form
const Form = () => {
 const [formData, setFormData] = useState({ name: '', email: '' });

 const handleChange = useCallback(
  (e) => setFormData({ ...formData, [e.target.name]: e.target.value }),
  [formData]
 );
```

```
  return (
    <form>
      <InputField label="Name" value={formData.name}
onChange={handleChange} />
      <InputField label="Email" value={formData.email}
onChange={handleChange} />
    </form>
  );
};
```

---

60. You have a React component that performs an expensive calculation based on props. Every time a parent component re-renders, this calculation runs again, leading to performance issues. How would you prevent this?

Answer:

Use useMemo() to memoize the result of the expensive calculation and recalculate it only when the relevant props change.

```
const ExpensiveComponent = ({ prop1 }) => {
  const expensiveCalculation = useMemo(() => {
    // expensive operation
    return calculateResult(prop1);
  }, [prop1]);

  return <div>{expensiveCalculation}</div>;
};
```

This ensures the expensive calculation is only run when prop1 changes.

---

61. You notice that your React application

experiences performance issues because it frequently re-renders parts of the UI unnecessarily. How would you diagnose and optimize the app to avoid these issues?

Answer:

To diagnose the issue, you would:

Use **React DevTools Profiler** to identify which components are re-rendering unnecessarily.

Identify and eliminate prop drilling and excessive re-renders by optimizing with **React.memo(), useCallback(), and useMemo().**

Consider lazy-loading components that are not immediately needed using **React.lazy() and Suspense**.

Optimization example with React.memo():

```
const ChildComponent = React.memo(({ data }) => {
  // Only re-renders when `data` changes
  return <div>{data}</div>;
});
```

---

62. You are building a real-time chat application where messages are updated frequently. Each new message causes the entire chat window to re-render, leading to a performance bottleneck. How would you prevent expensive state updates and improve performance?

Answer:

To prevent expensive state updates, use immutable updates and **useRef()** to manage the chat state efficiently. You can also optimize re-renders by using **React.memo()** for individual message components.

```
const ChatMessage = React.memo(({ message }) => {
  return <div>{message}</div>;
});

const ChatWindow = ({ messages }) => {
  return (
    <div>
      {messages.map((msg, index) => (
        <ChatMessage key={index} message={msg} />
      ))}
    </div>
  );
};
```

Additionally, you can append new messages using functional state updates to ensure state changes are efficient.

```
setMessages((prevMessages) => [...prevMessages, newMessage]);
```

---

63. You are developing a gallery application with a grid of images, but the app suffers from slow load times due to large image files. How would you optimize the performance of rendering and loading images?

Answer:

You can optimize image loading by:

Lazy loading images using **loading="lazy"** attribute.

Implementing image compression and serving responsive images with different resolutions using srcset.

Using skeleton loaders or placeholders to improve user experience.

```
const ImageComponent = ({ src, alt }) => {
 return <img src={src} alt={alt} loading="lazy" />;
};
```

To handle various screen sizes and image resolutions, use the **srcset** attribute:

```
<img
 src="image-400w.jpg"
 srcset="image-400w.jpg 400w, image-800w.jpg 800w"
 sizes="(max-width: 600px) 400px, 800px"
 alt="Gallery"
/>
```

---

64. You have a deeply nested component tree where state is passed down multiple levels through props. The entire tree re-renders when the state changes, even if the change only affects a small part of the UI. How would you prevent unnecessary re-renders?

Answer:

One way to solve this problem is by using React.Context to avoid prop drilling and only update the parts of the UI that depend on the changed state. Another approach is to use local state for components that don't need the entire state tree, and use React.memo() for components that don't need to re-render.

```
const MyContext = React.createContext();

const ParentComponent = () => {
 const [state, setState] = useState({ data: 'initial', updateData: 'new' });

 return (
  <MyContext.Provider value={state}>
```

```
    <ChildComponent />
   </MyContext.Provider>
  );
};

const ChildComponent = React.memo(() => {
  const { data } = useContext(MyContext);

  return <div>{data}</div>; // Only re-renders when `data` changes
});
```

---

65. You are building a dynamic form builder where fields are dynamically added or removed. Re-rendering the entire form each time a field is added or removed causes performance degradation. How would you optimize it?

Answer:

<u>You can optimize this by:</u>

Using **key** to uniquely identify form fields and avoid unnecessary re-renders.

Employing **React.memo()** for individual form fields.

Batch updates for state changes using unstable_batchedUpdates to avoid multiple re-renders during state updates.

<u>Example of optimizing the form with React.memo():</u>

```
const Field = React.memo(({ fieldData, onChange }) => {
  return (
    <input
      value={fieldData.value}
      onChange={(e) => onChange(e.target.value)}
    />
  );
});
```

```
const DynamicForm = ({ fields, updateField }) => {
 return (
  <form>
   {fields.map((field, index) => (
    <Field key={field.id} fieldData={field} onChange={(value) =>
updateField(index, value)} />
   ))}
  </form>
 );
};
```

By memoizing each field, you avoid re-rendering the entire form when only one field changes.

---

66. In a server-side rendered (SSR) React application, you're fetching dynamic content during rendering. However, server response times are slow, affecting the user experience. How would you optimize the page load while maintaining SEO benefits?

Answer:

One approach is to use client-side hydration with placeholders for dynamic content. Render the static content server-side, and then fetch dynamic content after the initial render using useEffect() or SWR (stale-while-revalidate) for fetching data.

```
const Page = ({ staticData }) => {
 const { data: dynamicData, error } = useSWR('/api/dynamic-content',
fetcher, {
  fallbackData: staticData,
 });

 if (error) return <div>Error loading dynamic content</div>;
 return <div>{dynamicData || 'Loading dynamic content...'}</div>;
};
```

---

67. Should a ref created with useRef be added to the dependency array of useEffect?

Answer:

No, refs created with useRef should not be added to the dependency array of useEffect. This is because refs do not trigger re-renders when their .current property changes. The purpose of the useEffect dependency array is to track changes in values that affect re-rendering, like state or props. Since ref updates do not cause re-renders, adding them to the dependency array is unnecessary.

For example:

```
const myRef = useRef(null);

useEffect(() => {
  // Code using myRef.current
}, []); // No need to add myRef to the dependency array
```

In rare cases, if you want to track changes in the mutable object the ref points to, you may manually handle that, but it's uncommon.

---

68. What happens if you update a state using the current state directly without using the previous state?

Answer:

Updating state directly based on the current state can lead to incorrect results due to the asynchronous nature of state updates.
If multiple state updates occur in quick succession, React may batch updates, and the current state may not reflect the latest

value. It's safer to use the function form of setState, which ensures you have the latest state.

```
// Incorrect:
setCount(count + 1);
setCount(count + 1); // This might result in just +1 instead of +2

// Correct:
setCount((prevCount) => prevCount + 1);
setCount((prevCount) => prevCount + 1); // This ensures +2
```

---

69. What are the limitations of using useEffect for fetching data?

Answer:

Using useEffect for fetching data can lead to race conditions if multiple effects trigger due to fast user interactions or rerenders.

It also doesn't handle async functions directly as useEffect expects the returned value to be a cleanup function.

Another issue is that the effect may fire multiple times during the same render if dependencies are not correctly set.

To handle async fetches properly, make sure to:

1. Use async functions inside useEffect.
2. Cancel previous fetch requests to avoid race conditions.

```
useEffect(() => {
 let isMounted = true;
 async function fetchData() {
   const result = await fetch('api-url');
   if (isMounted) {
     setData(result);
```

```
    }
  }
  fetchData();
  return () => {
    isMounted = false; // Cleanup to prevent updating unmounted
components
  };
}, []);
```

---

70. You need to make three different API calls when a component mounts. The component should only render after all API responses are received. How would you implement this in React?

Answer:

You can use Promise.all() to handle multiple API calls in parallel. This ensures that all API calls are completed before updating the state, triggering the render.

```
useEffect(() => {
  const fetchData = async () => {
    try {
      const [response1, response2, response3] = await Promise.all([
        fetch('api/endpoint1'),
        fetch('api/endpoint2'),
        fetch('api/endpoint3'),
      ]);

      const data1 = await response1.json();
      const data2 = await response2.json();
      const data3 = await response3.json();

      // Set the combined data to state
      setData([data1, data2, data3]);
    } catch (error) {
      console.error('Error fetching data', error);
    }
  };

  fetchData();
}, []);
```

71. You have a list of 10,000 items in your component, and each time you update the state, the component re-renders and lags the UI. How would you optimize performance to handle this large list efficiently?

Answer:

To handle large lists efficiently, you can:

Use React.memo() to prevent unnecessary re-renders.

Implement virtualization (e.g., using react-window or react-virtualized) to only render the items currently in view.

Use useMemo() to memoize the list if its data doesn't change frequently.

Example:

```
import { FixedSizeList as List } from 'react-window';

const ItemList = React.memo(({ items }) => (
  <List
    height={400}
    itemCount={items.length}
    itemSize={35}
    width={300}
  >
    {({ index, style }) => (
      <div style={style}>
        {items[index]}
      </div>
    )}
  </List>
));
```

72. You are fetching data in a useEffect hook, but when the user quickly navigates away from the page, the state is updated even though the component is unmounted, causing a memory leak. How would you prevent this?

Answer:

To prevent memory leaks, you can use a cleanup function within useEffect to cancel any pending asynchronous actions. Set a flag to determine if the component is still mounted.

```
useEffect(() => {
 let isMounted = true;

 async function fetchData() {
  const result = await fetch('api-url');
  if (isMounted) {
   setData(result);
  }
 }

 fetchData();

 return () => {
  isMounted = false; // Cleanup when the component unmounts
 };
}, []);
```

73. You have a dashboard where different users have different permissions, and certain components should only render if the user has the appropriate permissions. How would you handle this conditional rendering?

Answer:

You can create a higher-order component (HOC) or a utility function to check user permissions and conditionally render the component.

```
const withPermission = (Component, requiredPermission) => {
  return (props) => {
    const { user } = useContext(UserContext);
    return user.permissions.includes(requiredPermission) ? <Component
{...props} /> : null;
  };
};
// Usage
const AdminPanel = withPermission(AdminComponent, 'admin');
```

---

74. You have an effect that depends on an array of items, and every time the array updates, it triggers an API call. The problem is that the effect is triggering continuously, causing an infinite loop. How would you prevent this?

Answer:

To prevent the infinite loop, ensure that the dependency array for useEffect contains only the necessary dependencies and doesn't include values that get updated within the effect itself. You might also want to use a stable reference (like useMemo) for complex dependencies or introduce a flag to prevent the effect from running unnecessarily.

```
const [data, setData] = useState([]);
const memoizedItems = useMemo(() => items, [items]);

useEffect(() => {
  const fetchData = async () => {
    const result = await apiCall(memoizedItems);
    setData(result);
  };
  fetchData();
}, [memoizedItems]); // Only trigger when memoizedItems changes
```

75. You want to improve your app's performance by lazy loading certain components that are not immediately needed when the app loads. How would you implement this?

Answer:

React provides React.lazy() and Suspense to lazy load components. You can wrap the dynamically imported component with React.lazy() and use Suspense to display a fallback while the component is loading.

```
const LazyComponent = React.lazy(() => import('./LazyComponent'));
function App() {
 return (
  <Suspense fallback={<div>Loading...</div>}>
   <LazyComponent />
  </Suspense>
 );
}
```

76. You have a large application where several components (at different levels in the hierarchy) need to read and modify the same piece of state. However, you cannot use Redux or Context. How would you solve this problem using only React?

Answer:

One approach is to use custom hooks to share state logic across multiple components. You can store the state in a module (as a singleton), and create a custom hook that allows components to access and modify the state.

```
// store.js
let state = {
```

```
  value: ''
};

const subscribers = [];

export const useSharedState = () => {
  const [, setLocalState] = useState(state);

  const setState = (newState) => {
    state = { ...state, ...newState };
    subscribers.forEach((cb) => cb(state));
  };

  useEffect(() => {
    const cb = (newState) => setLocalState(newState);
    subscribers.push(cb);

    return () => {
      subscribers.splice(subscribers.indexOf(cb), 1);
    };
  }, []);

  return [state, setState];
};

// In your component:
const [sharedState, setSharedState] = useSharedState();
```

---

77. You have a dynamic table where rows are added and removed frequently. The issue is that every time a new row is added or removed, the entire table is re-rendered, causing a significant performance hit. How would you optimize rendering?

Answer:

<u>You can optimize rendering by:</u>

Using **React.memo()** on each row component to prevent unnecessary re-renders.

Implementing **key-based rendering**, ensuring only the changed rows are updated.

Use the **useCallback()** hook to memoize the row addition/removal logic to avoid triggering unnecessary re-renders.

```
const Row = React.memo(({ rowData }) => {
 return <div>{rowData}</div>;
});

const Table = ({ data }) => {
 return (
   <div>
    {data.map((row, index) => (
      <Row key={row.id} rowData={row} />
    ))}
   </div>
 );
};
```

Also, virtualization can be used to render only the visible rows to improve performance.

---

78. You have a component that makes multiple asynchronous API calls to fetch user data when it mounts. However, the data can sometimes arrive out of order, causing inconsistent UI rendering. How would you handle this scenario?

Answer:

You can solve this problem by using a request versioning system or a cancellation mechanism to ensure the UI only updates with the latest API response.

Example using **AbortController** to cancel previous requests:

```
useEffect(() => {
 const controller = new AbortController();

 const fetchData = async () => {
  try {
   const response = await fetch('api-url', { signal: controller.signal });
   const data = await response.json();
   setUserData(data);
  } catch (error) {
   if (error.name === 'AbortError') {
    console.log('Request was canceled');
   }
  }
 };

 fetchData();

 return () => {
  controller.abort();  // Cancel the request when the component unmounts
or re-renders
 };
}, []);
```

---

79. You have two dropdowns, and the options in the second dropdown depend on the selection in the first dropdown. The problem is that users sometimes see stale options when they switch the selections quickly. How would you ensure that the second dropdown is always in sync with the first dropdown?

Answer:

The best way to handle this is to maintain a dependency relationship between the two dropdowns and clear the second dropdown's state when the first dropdown changes. You can also ensure the data fetching for the second dropdown is debounced.

```
const [dropdown1Value, setDropdown1Value] = useState('');
```

```
const [dropdown2Options, setDropdown2Options] = useState([]);

useEffect(() => {
  if (dropdown1Value) {
    const timeoutId = setTimeout(() => {
      fetch(`/api/options?filter=${dropdown1Value}`)
        .then((res) => res.json())
        .then((data) => setDropdown2Options(data));
    }, 300);  // Debounce to avoid quick switching issues

    return () => clearTimeout(timeoutId);  // Cleanup previous effect
  } else {
    setDropdown2Options([]);  // Reset options when first dropdown value
changes
  }
}, [dropdown1Value]);
```

---

80. You have an infinite scrolling list that loads more data as the user scrolls down. When new data is appended to the list, the scroll position jumps, disrupting the user experience. How would you maintain the scroll position while updating the list?

Answer:

You can store the current scroll position before updating the list and then restore it after rendering the new data. This can be done by accessing the scrollTop property of the list container.

```
const listRef = useRef(null);

const handleScroll = () => {
  const scrollPosition = listRef.current.scrollTop;

  // Fetch more data and append to the list
  setData((prevData) => [...prevData, ...newData]);

  // Restore the scroll position
  listRef.current.scrollTop = scrollPosition;
};
```

```
// JSX
<div ref={listRef} onScroll={handleScroll}>
 {data.map((item) => (
   <div key={item.id}>{item.name}</div>
 ))}
</div>
```

---

81. You have a component that receives props from a parent, but only some of the props change frequently. However, the child component is re-rendering every time the parent updates any props. How would you prevent unnecessary renders in the child component?

Answer:

You can use React.memo() to memoize the child component, ensuring it only re-renders when its relevant props change. Additionally, use useCallback() and useMemo() in the parent component to ensure stable references for functions or complex props.

```
const ChildComponent = React.memo(({ prop1, prop2 }) => {
 return <div>{prop1} - {prop2}</div>;
});

// Parent Component
const ParentComponent = () => {
 const stableProp1 = useMemo(() => computeProp1(), [dependency]);
 const memoizedCallback = useCallback(() => someCallback(),
[dependency]);

 return (
   <ChildComponent prop1={stableProp1} prop2={memoizedCallback} />
 );
};
```

---

82. You are using Server-Side Rendering (SSR) in Next.js to fetch data for a page, but due to network latency, the page load time is increased. How would you optimize this for a better user experience without losing SEO benefits?

Answer:

Cover below points:

Stale-while-revalidate (SWR): Use the SWR technique where you return stale data initially and revalidate in the background for the latest data. This speeds up the page load.

Preload critical data: Fetch the most important data first, while deferring secondary data requests to run after the initial render.

Example using getServerSideProps() with SWR:

```
export async function getServerSideProps() {
  const data = await fetch('api-endpoint').then(res => res.json());

  return { props: { initialData: data } };
}

const Page = ({ initialData }) => {
  const { data } = useSWR('/api-endpoint', fetcher, { fallbackData:
initialData });

  return <div>{data ? renderData(data) : 'Loading...'}</div>;
};
```

---

83. You have multiple API calls that need to be fired concurrently, but you want to display the results in the order the requests were made, not the order in which the responses return. How would you handle this?

Answer:

You can make concurrent requests using Promise.all() but map each response back to its respective request to maintain order. This ensures the results are processed and displayed in the correct order.

```
useEffect(() => {
  const fetchData = async () => {
    const requests = [fetch('api/endpoint1'), fetch('api/endpoint2')];
    const responses = await Promise.all(requests);

    const orderedData = await Promise.all(
      responses.map((response) => response.json())
    );

    setData(orderedData);
  };

  fetchData();
}, []);
```

---

84. Can you explain the architecture of React Fiber and how it improves rendering performance compared to the previous reconciliation algorithm?

Answer:

React Fiber is **an internal rewrite of React's core reconciliation engine, introduced in React 16** (2017). It's not a separate library or feature you use directly, but rather an upgraded architecture that makes React faster, more efficient, and capable of handling complex updates.
In simple terms, **React Fiber improves how React updates the UI by breaking the work into smaller chunks.** Before Fiber, React processed updates synchronously in one big pass, which could slow down apps with heavy computations or animations. Fiber allows React to pause, prioritize, or even abort work as

needed, making updates smoother and more responsive.

Main features:
• Incremental Rendering: Splits rendering into manageable pieces, letting React work on high-priority tasks (like user interactions) first.
• Better Scheduling: Prioritizes updates (e.g., animations over background tasks) using a system inspired by requestIdleCallback.
• Concurrency: Enables features like React Suspense and Concurrent Mode, where React can work on multiple tasks without blocking the main thread.

Fiber makes React feel snappier, especially for large apps or animations, by avoiding long freezes. It's the foundation for modern React features, though as a developer, you don't interact with it directly—just benefit from its improvements!

---

85. What are micro frontends, and how do you implement them in a React application? What challenges might arise?

Answer:

Micro frontends involve breaking a large web application into smaller, independently deployable applications, allowing different teams to develop and maintain them separately. This architectural approach enhances flexibility and scalability. Implementation in a React application can be achieved through the following methods:

Module Federation: Utilize Webpack's Module Federation feature to allow different micro frontend applications to share components and libraries dynamically at runtime. This enables teams to work on separate projects while reusing components efficiently.

Single SPA: Use Single SPA to orchestrate multiple micro frontends within a single application. This framework allows various micro frontends to coexist on the same page, enabling route-based loading and independent deployments.

Routing: Each micro frontend can handle its routing independently, while a root application can aggregate these routes, providing a cohesive user experience. For example, using React Router within each micro frontend ensures isolated navigation.

Challenges of implementing micro frontends include:

Shared State Management: Coordinating state between micro frontends can be complex, particularly if they need to share data or user context. A centralized state management solution, such as Redux or context providers, may be necessary.

Versioning Conflicts: With multiple teams deploying independently, ensuring that shared libraries and dependencies are compatible can lead to conflicts. Versioning strategies must be established to mitigate these issues.

Performance Overhead: Micro frontends can introduce loading times if not optimized. Lazy loading components and ensuring each micro frontend is as lightweight as possible is crucial to maintain performance.

By addressing these challenges, micro frontends can significantly improve development efficiency and maintainability in large applications.

---

86. What are synthetic events in React, and how do they differ from native browser events? Why is this important for performance?

Answer:

Synthetic events are **React's cross-browser wrapper around native events.** They provide a consistent API that behaves identically across different browsers, addressing compatibility issues.

```
function Button() {
  const handleClick = (event) => {
    console.log(event.target); // SyntheticEvent, points to the button
  };

  return <button onClick={handleClick}>Click me</button>;
}
```

Here, event is a synthetic event, not the raw browser event. It has properties like target, type, or preventDefault(), but they're React's version.

Key differences between synthetic and native events include:

Normalization: Synthetic events normalize event properties, ensuring that methods like event.preventDefault() and event.stopPropagation() behave consistently across different browsers. This reduces the complexity of handling browser-specific event behaviors.

Pooling: React implements event pooling, meaning that it reuses synthetic event objects to reduce memory usage and garbage collection overhead. After an event callback is executed, the synthetic event is nullified, which prevents memory leaks. This is crucial for performance, especially in applications with numerous event handlers.

Automatic Binding: In React, event handlers are automatically bound to the component instance, eliminating the need for manual binding in class components.

The use of synthetic events enhances performance by minimizing the overhead associated with event handling, providing a smoother and more efficient user experience.

87. What are some of the key features introduced in React 18, and how do they improve the developer experience and application performance?

Answer:

React 18 introduces several significant features that enhance both the developer experience and application performance:

1. Concurrent Rendering: This feature allows React to work on multiple tasks simultaneously, improving the overall responsiveness of the application. It enables React to pause rendering work, handle high-priority updates first, and resume less urgent tasks, resulting in smoother user interactions.

2. Automatic Batching: React 18 enables automatic batching of state updates. Previously, only updates within event handlers were batched, but now multiple state updates from asynchronous functions are batched together, reducing the number of re-renders and improving performance.

3. Suspense for Data Fetching: This allows components to "wait" for asynchronous data before rendering. Developers can use Suspense to show a loading indicator while the data is being fetched, leading to better user experiences and avoiding loading spinners scattered throughout the app.

4. useTransition Hook: This hook allows developers to mark updates as non-urgent. By using startTransition, developers can indicate that a particular update is not critical, allowing React to keep the UI responsive during large updates. This is particularly useful for transitions and animations.

5. useDeferredValue Hook: This new hook helps to defer updating a value until the rendering is less busy. For example, it can be used to keep an input value up-to-date while deferring updates to a related state, improving performance during heavy renders.

6. useId Hook: This hook generates unique IDs for components, which is particularly useful for accessibility. It ensures that IDs are unique across server and client rendering, avoiding hydration mismatches.

---

88. Ever hovered over a Chrome tab and noticed memory usage HIGH in your React app? Is it due to heavy/ big data inReact state ?

Answer:

Whether via useState, useReducer, or this.setState , it is just JavaScript.
It's stored in the V8 JavaScript heap (the memory space managed by Chrome's JS engine).

State is stored in closures or React's internal Fiber nodes.
It's part of the heap memory, not in the DOM or stack.
It's not automatically garbage-collected if lingering references (like in event handlers) exist.

Now, What Does Chrome's Memory Usage Show?
When you hover over a Chrome tab or use the Task Manager:
You're seeing the total memory used by that tab's process.

This includes:
JavaScript heap (React state, closures, etc.)
DOM & layout trees
Images, fonts, styles
Browser internals and GC overhead

So yes, React state is part of that memory but it's just one part of a much bigger picture.

Now, how to Debug?

Use Chrome DevTools:
Memory tab → Heap Snapshot → search your state
Performance tab → monitor JS heap over time
Look out for retained closures or detached DOM nodes (common memory leaks!)

In short :
Chrome memory ≠ just React state
React state lives in the JavaScript heap

---

5 React.js Custom Hooks You Must Practice Before Going for an Interview

When preparing for a React.js interview, mastering core concepts like state management, lifecycle methods, and component optimization is essential. However, understanding custom hooks can give you a significant edge, especially for senior roles. Custom hooks allow developers to encapsulate reusable logic, making applications more maintainable and modular. In interviews, showcasing how you create and leverage custom hooks can demonstrate deep React proficiency.

Here are five custom hooks that you must practice before walking into your next React interview.

1. useFetch: Fetching Data Efficiently

Fetching data is one of the most common operations in modern web applications. The useFetch custom hook allows you to reuse data-fetching logic across multiple components.

Implementation:

```
import { useState, useEffect } from 'react';

const useFetch = (url) => {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch(url);
        const result = await response.json();
        setData(result);
      } catch (error) {
        setError(error);
      } finally {
        setLoading(false);
      }
    };
    fetchData();
  }, [url]);

  return { data, error, loading };
};
```

Key Benefits:

1. Reusable for any API endpoint.
2. Handles loading and error states.
3. Simplifies fetching logic in your components.

Example Usage:

```
const UsersList = () => {
  const { data, error, loading } =
useFetch('https://api.example.com/users');

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error loading data!</p>;
```

```
  return (
    <ul>
      {data.map(user => <li key={user.id}>{user.name}</li>)}
    </ul>
  );
};
```

---

2. usePrevious: Tracking Previous State Values

There are times when you need to track the previous value of a state. usePrevious allows you to do that easily, a useful trick in scenarios like form validation or smooth UI transitions.

Implementation:

```
import { useEffect, useRef } from 'react';

const usePrevious = (value) => {
  const ref = useRef();

  useEffect(() => {
    ref.current = value;
  }, [value]);

  return ref.current;
};

export default usePrevious;
```

Key Benefits:

1. Keeps track of the previous value without side effects.
2. Useful for comparing state changes over time.

Example Usage:

```
const Counter = () => {
  const [count, setCount] = useState(0);
```

```
  const previousCount = usePrevious(count);

  return (
    <div>
      <p>Current Count: {count}</p>
      <p>Previous Count: {previousCount}</p>
      <button onClick={() => setCount(count +
1)}>Increment</button>
    </div>
  );
};
```

---

3. useDebounce: Debouncing Expensive Functions

Debouncing is essential when working with functions that trigger frequently, like a search input. The useDebounce hook delays execution until the user stops typing, preventing unnecessary API calls.

Implementation:

```
import { useState, useEffect } from 'react';

const useDebounce = (value, delay) => {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => clearTimeout(handler);
  }, [value, delay]);

  return debouncedValue;
};

export default useDebounce;
```

Key Benefits:

1. Reduces the number of executions of a function.
2. Ideal for optimizing input-heavy components like search fields.

Example Usage:

```
const Search = () => {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 500);

  useEffect(() => {
    if (debouncedQuery) {
      // Trigger API search here
      console.log('Searching for:', debouncedQuery);
    }
  }, [debouncedQuery]);

  return <input value={query} onChange={(e) =>
setQuery(e.target.value)} placeholder="Search..." />;
};
```

---

4. useToggle: Toggling Booleans

Many components in React require toggling boolean states, such as showing/hiding modals or switching themes. useToggle abstracts this logic into a simple, reusable hook.

Implementation:

```
import { useState } from 'react';

const useToggle = (initialValue = false) => {
  const [state, setState] = useState(initialValue);
  const toggle = () => setState(!state);

  return [state, toggle];
};
```

```
export default useToggle;
```

Key Benefits:

1. Simplifies toggling state logic.
2. Useful for modals, dropdowns, and light/dark themes.

Example Usage:

```
const ThemeToggle = () => {
  const [isDarkMode, toggleDarkMode] = useToggle();

  return (
    <div>
      <button onClick={toggleDarkMode}>
        {isDarkMode ? 'Switch to Light Mode' : 'Switch to Dark Mode'}
      </button>
    </div>
  );
};
```

---

5. useLocalStorage: Syncing State with LocalStorage

Persisting state between browser sessions can improve user experience, especially for preferences or settings. useLocalStorage is a custom hook that synchronizes state with localStorage.

Implementation:

```
import { useState } from 'react';

const useLocalStorage = (key, initialValue) => {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
```

```
    return initialValue;
  }
});

const setValue = (value) => {
  try {
    const valueToStore = value instanceof Function ?
value(storedValue) : value;
    setStoredValue(valueToStore);
    window.localStorage.setItem(key,
JSON.stringify(valueToStore));
  } catch (error) {
    console.error(error);
  }
};

return [storedValue, setValue];
};
```

Key Benefits:

1. Saves state between sessions.
2. Ideal for user preferences like themes or language settings.

Example Usage:

```
const Settings = () => {
  const [theme, setTheme] = useLocalStorage('theme', 'light');

  return (
    <div>
      <p>Current Theme: {theme}</p>
      <button onClick={() => setTheme(theme === 'light' ? 'dark' :
'light')}>
        Toggle Theme
      </button>
    </div>
  );
};
```

Chapter 2: Mastering JavaScript

1. Define Javascript.

Answer:

JavaScript is a
• High-level – abstracts low-level memory management,
• Async – supports non-blocking operations using callbacks, promises, and async/await,
• Single-threaded – uses one thread with an event loop for concurrency,
• Prototypal inherited – uses prototype chains for inheritance instead of classical classes,
• Garbage-collected – memory is managed automatically,
• Interpreted, but JIT compiled – parsed and optimized at runtime by modern engines,
• Multi-paradigm – supports object-oriented, functional, and imperative styles,
• Dynamic – types are determined at runtime, allowing flexible coding.

---

2. Explain prototypal inheritance.

Answer:

If you are asked this question cover these points 👇

1. What is Prototypal Inheritance?

JavaScript allows objects to inherit properties and methods directly from other objects via a chain of prototypes
- this is called prototypal inheritance.

2. What is __proto__?

Every object has an internal property called __proto__ (or [[Prototype]]), which points to the object it inherits from.

```
const animal = { eats: true };
const rabbit = Object.create(animal);
console.log(rabbit.__proto__ === animal); // true
```

3. What is .prototype?

.prototype is a property of constructor functions only.

It’s used to define shared methods for all instances created using the constructor.

```
function Animal() {}
Animal.prototype.walk = function() { console.log("walking"); };

const dog = new Animal();
dog.walk(); // "walking"
```

4. Prototype Chain

When you access a property:
JS first looks at the object
Then its __proto__
And keeps climbing up until it finds the property or reaches null.

rabbit -> animal -> Object.prototype -> null

5. How It Looks in the Browser Console

```
const animal = { eats: true };
const rabbit = Object.create(animal);
rabbit.jumps = true;
```

```
console.log(rabbit);
```

Expanding in DevTools shows:

```
{
  jumps: true
  [[Prototype]]: {
    eats: true
    [[Prototype]]: Object
  }
}
```

__proto__ is a developer-accessible alias for the internal [[Prototype]] property.

6. Modern Approach – ES6 Classes

ES6 classes are just syntactic sugar over JavaScript's prototypal inheritance system
Classes use prototypal inheritance under the hood.

```
class Animal {
  eat() { console.log("eats"); }
}

const cat = new Animal();
http://cat.eat(); // "eats"
```

7. Benefits

Memory efficient (methods shared across instances)
Dynamic: you can modify prototype at runtime

8. Drawbacks

Debugging prototype chains can be tricky
Confusing for those coming from classical OOP backgrounds

3. What is a closure?

Answer:

A closure happens when:

A function is defined inside another function, and
The inner function remembers the variables from the outer function even after the outer function has finished running.

Example:

```
function outer() {
  let count = 0;

  function inner() {
    console.log(count);
  }
  return inner;
}

const fn = outer();
fn(); // prints 0
```

Even though outer() is done, inner() still has access to count. That's a closure.

Why and where are closures useful?

Data privacy: Variables inside a function can be hidden and accessed only via closures.

Maintain state: Useful for things like counters or caching.

Asynchronous code: setTimeout or event handlers use closures to remember variables.

Functional programming: Enables powerful patterns like memoization, debounce, throttle, etc.

Additional point you can add :
Closures use lexical scope to "remember" variables. The inner function remembers the variables from the outer scope where it was lexically defined, even if that outer function has already finished executing.

---

4. JavaScript is lexically scoped but what is Lexical Scope ?

Answer:

Lexical scope means a variable's scope is determined by where it's written in the code - not where it's called.

It's all about location in your code.

It’s how closures, setTimeout, useEffect, and more advanced patterns work.

JavaScript is lexically scoped, not dynamically scoped.

That means:
The scope of variables is determined by where the code is written (at compile time).
Functions remember the environment where they were defined, not where they are called.
Dynamic scope would mean variables are resolved based on the call stack at runtime, which JavaScript does not do.

Example:

```
function outer() {
  let x = 10;

  function inner() {
    console.log(x); // Can access 'x' from outer
  }

  inner();
}
outer();
```

Here, inner() can access x because it was defined inside outer().
This is lexical scope - the function "remembers" the environment where it was created.

Key Points:

Lexical scope is static (fixed at code definition).

Inner functions can access variables from their outer scopes.

It's the foundation of closures in JavaScript.

---

5. Why Debounce in JavaScript Needs Closures ?

Answer:

It’s all thanks to closures - one of JavaScript’s most powerful (and often underrated) features.

What is debounce?
Debounce ensures a function runs only once after a pause, no matter how many times it's triggered.

Use cases:

Search box suggestions
Resize event optimization
Button spam prevention

Why Closures Matter in Debounce

```
function debounce(func, delay) {
  let timer; // ← closure in action

  return function (...args) {
    clearTimeout(timer); // reset old timer
    timer = setTimeout(() => {
      func.apply(this, args);
    }, delay);
  };
}
```

timer is defined once inside debounce and remembered by the returned function, even after debounce() is done executing. This “memory” is what we call a closure - it keeps timer alive between function calls.

The timer variable must persist between invocations ,closures let us store that state cleanly inside the returned debounced function.

---

6. Explain Event Loop.

Answer:

Cover these 6 points :

1. Event Loop is a mechanism that handles asynchronous callbacks in single-threaded environments like JavaScript.

2. It enables non-blocking I/O by managing the call stack, macrotask queue, and microtask queue.

3. Key components:

Call Stack: Executes functions

Web APIs: Handle async tasks like setTimeout, DOM events

Macrotask Queue (aka Task Queue): Queues callbacks from Web APIs like setTimeout, setInterval, I/O

Microtask Queue: Queues callbacks from Promise.then, queueMicrotask, process.nextTick

4. Event loop continuously checks the call stack and pushes tasks from queues when the stack is empty.

5. Execution order: microtasks run before macrotasks. Example: Promise.then runs before setTimeout.

6. Macrotasks include setTimeout, setInterval, and I/O tasks; microtasks include Promise, process.nextTick, MutationObserver.

---

7. Difference between Iterable and Enumerable in Javascript?

Answer:

What is an Iterable?

An iterable is something that can be looped using a for...of loop or spread with ....

To be iterable, an object must implement the special method: [Symbol.iterator]()

Examples of common built-in iterables:

Arrays
Strings
Maps
Sets

Example: Arrays are Iterable

```
const arr = ['x', 'y', 'z'];
for (const value of arr) {
  console.log(value); // x, y, z
}
console.log([...arr]); // ['x', 'y', 'z']
```

The array has a built-in [Symbol.iterator]() method, so for...of and spread work perfectly.

What is Enumerable?

An enumerable object has properties that can be listed using for...in or Object.keys().

Most plain objects and arrays have enumerable properties.

Example: Objects are Enumerable

```
const obj = { a: 1, b: 2 };
for (const key in obj) {
  console.log(key); // a, b
```

}

console.log(Object.keys(obj)); // ['a', 'b']

Note : Arrays are both enumerable and iterable

---

8. What is the difference between null and undefined in JavaScript?

Answer:

null is an assignment value. It is used when we want to intentionally leave a variable empty. It is an object type.

undefined means a variable has been declared but has not yet been assigned a value. It is a type itself.

Example:

```
let x; // undefined (not initialized)
let y = null; // null (explicitly no value)
let obj = { a: null };

console.log(x); // undefined
console.log(y); // null
console.log(obj.a); // null
console.log(obj.b); // undefined (property doesn't exist)
console.log(x == y); // true (loose equality)
console.log(x === y); // false (strict equality)
```

---

9. Give an example of closures in JavaScript?

Answer:

A closure in JavaScript is **a function that retains access to its outer (lexical) scope's variables even after the outer function has finished executing.**
Essentially, it "closes over" the variables from its parent scope, allowing the inner function to remember and access those variables later. Closures are created every time a function is defined inside another function.

Closures are powerful because they enable:

- Data privacy (encapsulation).
- Persistent state across function calls.
- Functional programming patterns (like currying or memoization).

```
function createCounter() {
  let count = 0; // This variable is in the outer function's scope

  return function increment() {
    count++; // Inner function accesses outer scope's 'count'
    return count;
  };
}

const counter1 = createCounter();
const counter2 = createCounter();

console.log(counter1()); // 1
console.log(counter1()); // 2
console.log(counter2()); // 1 (separate closure, separate 'count')
console.log(counter2()); // 2
```

createCounter defines count and returns increment, which closes over count.counter1 and counter2 each have their own count (closure), so they track separate counts.
increment retains access to count, incrementing it each call.

---

10. What is the difference between call(), apply(), and bind()?

Answer:

call(): Invokes a function with a specified this context and **arguments passed individually.**

apply(): Invokes a function with a specified this context and **arguments passed as an array**.

bind(): Returns a new function with a specified this context and optional arguments, which **can be invoked later.**

Method	Purpose	How Arguments Are Passed	Execution	Returns
`call()`	Calls the function with a specific `this` value and arguments individually.	Arguments passed as a comma-separated list.	Executes immediately.	Function's return value.
`apply()`	Calls the function with a specific `this` value and arguments as an array.	Arguments passed as an array.	Executes immediately.	Function's return value.
`bind()`	Creates a new function with a specific `this` value and optional preset arguments.	Arguments passed as a comma-separated list.	Does not execute immediately; returns a new function.	New bound function.

Example:

```
const hero = {
  name: "Batman",
};

function greet(greeting, city) {
  return `${greeting}, I am ${this.name} from ${city}!`;
}

// Using call()
console.log(greet.call(hero, "Hello", "Gotham")); // Hello, I am Batman from Gotham!

// Using apply()
console.log(greet.apply(hero, ["Hi", "Gotham"])); // Hi, I am Batman from Gotham!

// Using bind()
const boundGreet = greet.bind(hero, "Hey"); // Preset greeting
console.log(boundGreet("Gotham")); // Hey, I am Batman from Gotham!
```

call(): Immediately calls greet with hero as this, passing "Hello" and "Gotham" as individual arguments.

apply(): Immediately calls greet with hero as this, passing arguments as an array ["Hi", "Gotham"].
bind(): Creates a new function boundGreet with hero as this and "Hey" preset as the first argument. When called later with "Gotham", it completes the arguments.

---

11. What is the difference between == and === in JavaScript?

Answer:

== (abstract equality) compares two values for equality, but performs type coercion if the types differ.

=== (strict equality) compares both the values and types, without coercion.

Example:

```
console.log(5 == '5'); // true (type coercion occurs)
console.log(5 === '5'); // false (no type coercion)
```

---

12. What are the main differences between var, let, and const?

Answer:

var:
Function-scoped.
Can be re-declared and updated.
Subject to hoisting.

let:
Block-scoped.

Cannot be re-declared within the same scope, but can be updated.
Not initialized until its declaration is evaluated.

const:
Block-scoped.
Cannot be re-declared or updated.
Must be initialized at the time of declaration.

Feature	`var`	`let`	`const`
Scope	Function-scoped (or global if outside a function).	Block-scoped (limited to `{}` block).	Block-scoped (limited to `{}` block).
Hoisting	Hoisted to the top of its scope and initialized as `undefined`.	Hoisted but not initialized (Temporal Dead Zone).	Hoisted but not initialized (Temporal Dead Zone).
Reassignment	Can be reassigned.	Can be reassigned.	Cannot be reassigned (but object properties can be mutated).
Redeclaration	Can be redeclared in the same scope.	Cannot be redeclared in the same scope.	Cannot be redeclared in the same scope.
Use Case	Legacy; avoid due to scoping issues.	Variables that may change.	Constants or variables that shouldn't be reassigned.

Example:

```
// Hoisting and Temporal Dead Zone
console.log(v1); // undefined (hoisted, initialized as undefined)
console.log(l1); // ReferenceError: Cannot access 'l1' before initialization
console.log(c1); // ReferenceError: Cannot access 'c1' before initialization
var v1 = 1;
let l1 = 2;
const c1 = 3;

// Scope and Redeclaration in Loops
for (var i = 0; i < 2; i++) {
  var v2 = "var"; // Redeclared in same scope
  let l2 = "let"; // New binding per iteration
}
console.log(v2); // var (function-scoped, accessible outside)
console.log(l2); // ReferenceError: l2 is not defined (block-scoped)

// Reassignment and Mutation
const obj = { a: 1 };
obj.a = 2; // Allowed: mutating object
// obj = { b: 2 }; // TypeError: Assignment to constant variable
let x = 10;
x = 20; // Allowed: reassignment
var y = 30;
var y = 40; // Allowed: redeclaration
console.log(obj.a, x, y); // 2 20 40
```

---

13. What is 'this' keyword in JavaScript?

Answer:

The this keyword refers to the object from which the function is called. The value of this is determined by how a function is invoked:

<u>In a method:</u> this refers to the object the method belongs to.

<u>In a function (non-strict mode):</u> this refers to the global object (in strict mode, it's undefined).

<u>In an arrow function:</u> this is lexically inherited from the surrounding scope.

Example:

```
const obj = {
  name: 'John',
  greet() {
    console.log(this.name); // 'John'
  }
};
obj.greet();
```

---

14. What are template literals in JavaScript?

Answer:

Template literals are string literals allowing embedded expressions and multi-line strings. Introduced in ES6, they use backticks (`) instead of quotes.

Template literals allow for:

1. String interpolation: Embedding expressions inside strings using ${}.

2. Multi-line strings: Strings spanning multiple lines without the need for concatenation.

Example:

```
const name = 'Alice';
const greeting = `Hello, ${name}!`;
console.log(greeting); // Hello, Alice!

const multiLineString = `This is a
multi-line string`;
```

---

15. What is the difference between setTimeout

and setInterval in JavaScript.

Answer:

setTimeout: Executes a function after a specified delay (in milliseconds).

setTimeout(function, delay).

setInterval: Repeatedly calls a function at specified intervals (in milliseconds) until it is cleared.

setInterval(function, interval).

Example:

```
setTimeout(() => console.log('Executed once after 2 seconds'), 2000);
const intervalId = setInterval(() => console.log('Executed every 1 second'), 1000);

// clearInterval(intervalId); // To stop the interval
```

---

16. What are JavaScript modules?

Answer:

JavaScript modules allow developers to split code into separate files, making it easier to manage. Modules can import/export functionality.

Types of Javascript modules:

ES Modules (ESM): Modern standard, best for browsers and modern Node.js.

CommonJS: Traditional Node.js, synchronous, still widely used.

AMD: Older, browser-focused, async loading.

UMD: Compatibility layer for AMD/CommonJS.

Module	Syntax	Environment	Features	Loading
ESM	`import` / `export`	Browsers, Node.js	Static, tree-shaking	Async
CommonJS	`require()` / `exports`	Node.js	Dynamic, synchronous	Sync
AMD	`define()` / `require()`	Browsers (RequireJS)	Async loading	Async
UMD	Mixed AMD/CommonJS	Browsers/Node.js	Multi-environment	Depends

Example:

```
// ESM (math.js)
export const add = (a, b) => a + b;

// ESM (main.js)
import { add } from './math.js';
console.log(add(2, 3)); // 5

// CommonJS (math.js)
module.exports = { add: (a, b) => a + b };

// CommonJS (main.js)
const { add } = require('./math');
console.log(add(2, 3)); // 5
```

---

17. What is the difference between Object.create() and object constructors?

Answer:

Object.create():
Creates a new object with a specified prototype object and properties. It allows specifying the prototype explicitly.

Object constructors:
Use the new keyword to create a new instance from a constructor function.

Feature	`Object.create()`	Object Constructor
Purpose	Creates a new object with a specified prototype.	Creates a new object using a constructor function.
Prototype Setup	Directly sets the prototype to an existing object.	Uses the constructor's `prototype` property.
Property Definition	Optional second argument to define properties.	Properties defined inside the constructor.
Usage	More explicit for prototypal inheritance.	Traditional, class-like object creation.
Initialization	Doesn't execute a function; just links prototype.	Executes the constructor function to initialize.

Example:

```
// Using Object.create()
const proto = {
  greet() {
    return `Hello, I am ${this.name}!`;
  }
};

const obj1 = Object.create(proto); // obj1's prototype is proto
obj1.name = "Alice";
console.log(obj1.greet()); // Hello, I am Alice!

// Using Object Constructor
function Person(name) {
  this.name = name; // Initialize properties
}
Person.prototype.greet = function() {
  return `Hello, I am ${this.name}!`;
};

const obj2 = new Person("Bob"); // Creates new object, runs constructor
console.log(obj2.greet()); // Hello, I am Bob!

// Checking prototypes
console.log(Object.getPrototypeOf(obj1) === proto); // true
console.log(Object.getPrototypeOf(obj2) === Person.prototype); // true
```

---

18. What is the difference between debouncing and throttling?

Answer:

Debouncing:
Ensures that a **function is not called again until a certain amount of time** has passed without it being triggered. It's useful for events like typing or window resizing.

Throttling:
Ensures that a function **is called at most once in a specified period**, even if it is triggered multiple times.

Example:

```
function debounce(func, delay) {
  let timeoutId;
  return function(...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => func.apply(this, args), delay);
  };
}
```

How Throttle works:

1. let inThrottle: A flag to track if we're within the cooldown period (true = throttled).
2. Returned function: This is the throttled version of func.
3. if (!inThrottle): Only allow execution if we're not throttled.
4. func.apply(this, args): Calls func with the correct this context and arguments.
5. setTimeout: After limit milliseconds, it resets inThrottle to false, allowing the next call.

```
function throttle(func, limit) {
  let inThrottle;
  return function(...args) {
    if (!inThrottle) {
      func.apply(this, args);          // call the original function
      inThrottle = true;               // block further calls
      setTimeout(() => inThrottle = false, limit); // unblock after 'limit'
ms
    }
  };
}
```

---

19. Explain the difference between mutable and immutable objects in JavaScript.

Answer:

Mutable objects: Their state or content can be changed after they are created (e.g., objects, arrays).
Immutable objects: Once created, their state or content cannot

be changed (e.g., primitive values like strings and numbers).

Example:

```
let numbers = [1, 2, 3];
console.log(numbers); // [1, 2, 3]

// Modify the array
numbers.push(4);
numbers[0] = 10;
console.log(numbers); // [10, 2, 3, 4]
let greeting = "Hello";
console.log(greeting); // "Hello"

// Attempt to modify
greeting[0] = "Y"; // No effect
console.log(greeting); // "Hello"

// "Modification" creates a new string
let newGreeting = greeting.replace("H", "Y");
console.log(newGreeting); // "Yello"
console.log(greeting); // "Hello" (original unchanged)
```

---

20. What is the difference between shallow copy and deep copy in JavaScript?

Answer:

Shallow copy: Copies the reference to the original memory location. If the original object changes, the shallow copy reflects the changes.

Deep copy: Creates a completely independent copy of the object and its nested objects. Changes made to the original object will not affect the deep copy.

Shallow Copy: Spread operator (...), Object.assign(), slice() for arrays.

Deep Copy: JSON.parse(JSON.stringify()), structuredClone(), or

custom recursive functions.

Shallow Copy Example

```
// Original object with nested structure
let original = {
  name: "Alice",
  details: { age: 25, city: "London" }
};

// Shallow copy using spread operator
let shallowCopy = { ...original };

console.log("Original:", original);
// { name: "Alice", details: { age: 25, city: "London" } }
console.log("Shallow Copy:", shallowCopy);
// { name: "Alice", details: { age: 25, city: "London" } }

// Modify top-level property
shallowCopy.name = "Bob";
console.log("Original after name change:", original);
// { name: "Alice", details: { age: 25, city: "London" } }
console.log("Shallow Copy after name change:", shallowCopy);
// { name: "Bob", details: { age: 25, city: "London" } }

// Modify nested object
shallowCopy.details.age = 30;
console.log("Original after nested change:", original);
// { name: "Alice", details: { age: 30, city: "London" } }
console.log("Shallow Copy after nested change:", shallowCopy);
// { name: "Bob", details: { age: 30, city: "London" } }
```

The spread operator (...) creates a shallow copy.
Changing name (a top-level property) only affects the copy.
Changing details.age (a nested property) affects both, because details is a shared reference.

Deep Copy Example

```javascript
// Original object with nested structure
let original = {
  name: "Alice",
  details: { age: 25, city: "London" }
};

// Deep copy using JSON methods
let deepCopy = JSON.parse(JSON.stringify(original));

console.log("Original:", original);
// { name: "Alice", details: { age: 25, city: "London" } }
console.log("Deep Copy:", deepCopy);
// { name: "Alice", details: { age: 25, city: "London" } }

// Modify top-level property
deepCopy.name = "Bob";
console.log("Original after name change:", original);
// { name: "Alice", details: { age: 25, city: "London" } }
console.log("Deep Copy after name change:", deepCopy);
// { name: "Bob", details: { age: 25, city: "London" } }

// Modify nested object
deepCopy.details.age = 30;
console.log("Original after nested change:", original);
// { name: "Alice", details: { age: 25, city: "London" } }
console.log("Deep Copy after nested change:", deepCopy);
// { name: "Bob", details: { age: 30, city: "London" } }
```

JSON.parse(JSON.stringify()) creates a deep copy.
Changes to both top-level (name) and nested (details.age) properties in deepCopy do not affect original.

---

21. What are arrow functions and how do they differ from regular functions in JavaScript?

Answer:

Arrow functions are a concise way to write functions introduced in ES6. They differ from regular functions in several ways:

1. No this binding: Arrow functions do not have their own this context, they inherit this from the surrounding scope.

2. Implicit return: If the function body is a single expression, it can return the result implicitly without using the return keyword.

3. Cannot be used as constructors: Arrow functions cannot be used with new to create objects.

IMPORTANT: In JavaScript, the value of this depends on **how a function is called**, not where it's defined. But arrow functions change that behavior.

```
// ❌ Regular function (wrong `this`)
function Regular() {
  this.age = 0;
  setInterval(function () {
    this.age++;
    console.log('Regular:', this.age); // NaN, NaN, NaN, ...
  }, 1000);
}

// ✅ Arrow function (correct `this`)
function Arrow() {
  this.age = 0;
  setInterval(() => {
    this.age++;
    console.log('Arrow:', this.age); // 1, 2, 3, ...
  }, 1000);
}

// new Regular();
new Arrow();
```

---

22. What is Promise.all() in JavaScript.

Answer:

Promise.all() is a method that takes an iterable of promises (e.g., an array) and returns a single promise that resolves when all the promises in the iterable have resolved. If any of the promises reject, the resulting promise will be rejected immediately with the reason of the first promise that rejects.

Example:

```
const promise1 = Promise.resolve(10);
const promise2 = new Promise((resolve) => setTimeout(() => resolve(20), 1000));
const promise3 = new Promise((resolve) => setTimeout(() => resolve(30), 2000));

// Use Promise.all to wait for all of them to resolve
Promise.all([promise1, promise2, promise3])
  .then((results) => {
    console.log("All promises resolved:");
    console.log(results); // [10, 20, 30]
  })
  .catch((error) => {
    console.error("One of the promises failed:", error);
  });
```

---

23. What is the Symbol type in JavaScript, and how is it used?

Answer:

Symbol is a **primitive data type introduced in ES6**. Each **Symbol is a unique and immutable value** that can be used as an identifier for object properties. Symbols ensure that property keys are unique and cannot be accidentally overwritten.

- `Symbol` is a primitive, unique identifier.
- Great for safe property keys and advanced behaviors.
- Not enumerable, but accessible.
- Supports built-in JavaScript protocols (like iteration, toString, etc.).

```
// 1. Creating Symbols
const sym1 = Symbol("id");
const sym2 = Symbol("id");

console.log(sym1 === sym2); // false (unique values)

// 2. Using Symbols as object keys
const user = {
  name: "Alice",
  [sym1]: 123
};

console.log(user[sym1]); // 123

// 3. Symbol-keyed properties are hidden from typical enumeration
for (let key in user) {
  console.log(key); // "name" only, symbol is hidden
}

console.log(Object.keys(user)); // ["name"]
console.log(Object.getOwnPropertySymbols(user)); // [ Symbol(id) ]

// 4. Well-known Symbol: Symbol.iterator
const myIterable = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  }
};

for (let value of myIterable) {
  console.log(value); // 1, 2, 3
}
```

Every iterable in JavaScript — such as:
Arrays ([]), Strings ("abc"), Maps (new Map()), Sets (new Set()), TypedArrays, etc.
— has a **built-in [Symbol.iterator]() method**.
This method returns an iterator object, which lets them work with for...of, spread ..., and other iteration constructs.

---

24. What are generators in JavaScript?

Answer:

Generators in JavaScript are **special functions that can be paused and resumed**, giving you powerful control over execution flow.

What Makes a Function a Generator?

- Defined using **function*** syntax (note the *)
- Use **the yield keyword** to pause execution and return a value
- Can be resumed later using **.next()**

```
function* myGenerator() {
  yield 1;
  yield 2;
  yield 3;
}

const gen = myGenerator();

console.log(gen.next()); // { value: 1, done: false }
console.log(gen.next()); // { value: 2, done: false }
console.log(gen.next()); // { value: 3, done: false }
console.log(gen.next()); // { value: undefined, done: true }
```

Why Use Generators?

1. Lazy evaluation — generate data on the fly
2. Custom iterators — define how objects should be looped
3. Async control flow — easier stateful logic in loops (especially before async/await)

---

25. Explain the Object.freeze() method in JavaScript.

Answer:

Object.freeze(obj) prevents modifications to an object.

Once frozen:
✅ You can still read properties
❌ You can't add, delete, or change properties
❌ You can't reconfigure properties (e.g., change writable/enumerable flags)

Example:

```
const user = {
  name: "Alice"
};

Object.freeze(user);

user.name = "Bob";      // ❌ No effect
user.age = 30;          // ❌ No effect
delete user.name;       // ❌ No effect

console.log(user); // { name: "Alice" }
```

---

26. Implement memoization in Javascript?

Answer:

Memoization is a performance optimization technique where the result of a function call is cached based on its arguments, so subsequent calls with the same arguments return the cached result instead of recalculating.

```
function memoize(fn) {
  const cache = new Map();
  return function (...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) {
      return cache.get(key); // Return cached result
    }
    const result = fn(...args);
    cache.set(key, result); // Store result in cache
    return result;
  };
}
```

---

27. What are JavaScript module patterns, and how do you implement them in modern JavaScript?

Answer:

JavaScript modules allow developers to encapsulate functionality and create reusable, maintainable code.

Common module patterns include:

<u>IIFE (Immediately Invoked Function Expression):</u> It

encapsulates variables and exposes only necessary parts.

```
const myModule = (function () {
  const privateVar = 'secret';
  function publicMethod() {
    return privateVar;
  }
  return { publicMethod };
})();
console.log(myModule.publicMethod()); // Outputs 'secret'
```

ES6 Modules: The modern approach using export and import to structure code.

```
// module.js
export const myMethod = () => 'Hello World';

// main.js
import { myMethod } from './module.js';
console.log(myMethod()); // Outputs 'Hello World'
```

---

28. How do you handle circular dependencies in JavaScript modules?

Answer:

Circular dependencies occur when two or more modules depend on each other, leading to complex loading sequences or bugs. To handle circular dependencies:

Refactor the architecture: Break the dependency by introducing an intermediary module or service that both modules can depend on.

Use dynamic imports: Lazy load modules to resolve dependency issues.

Check module exports: Since JavaScript modules are hoisted, export statements are evaluated first, ensuring that at least the exported object is available. Example:

```
// Module A
import { B } from './moduleB.js';
export const A = () => `A depends on ${B()}`;

// Module B
import { A } from './moduleA.js';
export const B = () => `B depends on ${A()}`;
```

---

29. Explain the Event Loop's role in JavaScript concurrency. How would you handle long-running operations?

Answer:

JavaScript is **single-threaded**, meaning it runs one thing at a time. But it handles **asynchronous tasks** (like timers, network requests, etc.) using the **Event Loop**.

<u>How It Works :</u>

1. Call Stack – Where JS runs functions
2. Web APIs – Browser-provided APIs for async stuff (setTimeout, fetch, etc.)
3. Callback/Task Queue – Where completed async tasks wait
4. Event Loop – Keeps checking:

"Is the call stack empty? → Yes? Then push the next task from the queue."

```
Call Stack ← Event Loop ← Callback Queue ← Web APIs (Async Ops)
```

<u>For long-running tasks:</u>

Break into smaller chunks
Use Web Workers (browser)
Use async/await for async ops

---

30. How do you ensure high performance in large-scale JavaScript applications?

Answer:

Performance in large-scale applications can be maintained through multiple strategies:

Code-splitting: Split the application into smaller bundles to reduce initial load times.

Lazy loading: Defer the loading of non-critical resources until they're needed.

Tree-shaking: Eliminate unused code when bundling.

Memoization: Cache expensive computations using techniques like memoize in React or lodash's memoize function.

Debouncing/Throttling: Control how often functions are executed during frequent events like scrolling or resizing.

Reduce DOM manipulations: Minimize direct DOM access, and batch DOM updates using virtual DOM libraries like React.
Example:

---

31. What are the limitations of JavaScript and how do you overcome them in large-scale applications?

Answer:

JavaScript's limitations include:

Single-threaded nature: This can be overcome by using Web Workers for offloading tasks or service workers for caching and background processing.

Memory management: Memory leaks can occur due to closures, forgotten event listeners, or large object references. Use tools like Chrome DevTools to track memory usage and optimize large objects or arrays.

Global variables: Polluting the global namespace can lead to collisions. Use modules or closures to encapsulate variables and keep scope limited.

---

32. What is event delegation and how would you implement it in large DOM trees?

Answer:

Event delegation is a pattern where a single event listener is attached to a parent element to manage events for multiple child elements. This is efficient in cases of dynamically generated elements or large DOM structures, as it reduces the number of event listeners.

Instead of attaching an event to each child, the parent element handles events by checking the event target.
Example:

```
document.getElementById('parent').addEventListener('click', (event) => {
 if (event.target && event.target.matches('button')) {
   console.log('Button clicked:', event.target.textContent);
 }
});
```

This reduces the overhead of multiple event listeners and allows dynamic handling of new elements.

---

33. What are WeakMaps and WeakSets in JavaScript, and when would you use them?

Answer:

WeakMap and WeakSet are collections that allow references to objects without preventing garbage collection. These are used when you need to associate data with objects, but don't want to prevent those objects from being garbage collected.

WeakMap: Maps keys to values, where keys must be objects. If the key object has no other references, it can be garbage collected.

WeakSet: Stores unique objects. Once an object in a WeakSet has no other references, it can be garbage collected. Use case: Caching DOM elements in a dynamic web app where elements may be removed or added frequently. You can cache data without worrying about memory leaks:

```
const cache = new WeakMap();
function cacheData(element, data) {
  cache.set(element, data);
}
```

---

34. How would you implement a custom iterator in JavaScript?

Answer:

A custom iterator allows you to control how an object is iterated.
You need to implement the Symbol.iterator method, which returns an iterator object with a next() method.

Example of a custom iterator

```
const range = {
  start: 1,
  end: 3,
  [Symbol.iterator]() {
    let current = this.start;
    const end = this.end;
    return {
      next() {
        if (current <= end) {
          return { value: current++, done: false };
        }
        return { done: true };
      }
    };
  }
};

for (const num of range) {
  console.log(num); // 1, 2, 3
}
```

35. What are the common memory leaks in JavaScript, and how would you avoid them in large-scale applications?

Answer:

Memory leaks occur when objects are not properly garbage collected due to lingering references. Common causes include:

Global Variables: Unintentional global variables or excessive use of the global scope can prevent garbage collection. Use strict mode ('use strict') to catch undeclared variables.

Closures: Functions that reference variables outside their scope can cause memory leaks if those references persist longer than needed.

Detached DOM elements: If event listeners are attached to DOM elements that are removed, the memory remains allocated unless the event listeners are removed.

Circular References: Circular references between objects (especially DOM and JavaScript objects) can prevent garbage collection. To avoid memory leaks:

Use weak collections like WeakMap or WeakSet where possible to avoid retaining unnecessary object references.

Clean up event listeners and DOM references on component unmounts in frameworks like React.

Use proper scoping to prevent excessive use of closures.

---

36. How do generator functions differ from async functions? Provide a use case where generators would be preferable.

Answer:

Generator functions are **functions that can pause and resume execution**. They **return an iterator object** that can yield multiple values over time. Generators are **defined using the function*** syntax and can use the yield keyword to pause execution.

Async functions, on the other hand, **return a Promise** and are used to handle asynchronous code in a synchronous manner using await.
Example of a generator:

```
function* myGenerator() {
  yield 1;
  yield 2;
  yield 3;
}
```

```
const gen = myGenerator();
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
console.log(gen.next().value); // 3
```

A key difference is that generators do not inherently handle promises, whereas async/await are designed for promises. Generators are preferable in cases where you need more fine-grained control over the execution flow, such as implementing a custom iterator or complex control flow scenarios where execution is paused and resumed multiple times.

---

37. How does JavaScript's new keyword work under the hood, and how would you polyfill it?

Answer:

The new keyword in JavaScript performs the following steps:

1. Creates a new, empty object.

2. Sets the prototype of the new object to the constructor function's prototype.

3. Binds this to the new object inside the constructor function.

4. If the constructor returns an object, that object is returned; otherwise, the newly created object is returned.

<u>To polyfill the new keyword:</u>

```
function myNew(constructor, ...args) {
  const obj = Object.create(constructor.prototype);
  const result = constructor.apply(obj, args);
  return result instanceof Object ? result : obj;
}
```

---

38. Explain the concept of tail call optimization (TCO) in JavaScript. Does JavaScript natively support TCO?

Answer:

Tail Call Optimization (TCO) is an optimization that allows a recursive function to reuse the current stack frame for subsequent calls, rather than adding new stack frames for each call. This reduces the risk of stack overflow in deep recursion.

JavaScript engines, notably in strict mode and in compliant environments (like some versions of V8), support TCO when a function call is in tail position, meaning it's the last operation in the function.
Example:

```
'use strict';
function factorial(n, acc = 1) {
  if (n <= 1) return acc;
  return factorial(n - 1, n * acc); // Tail call
}
```

TCO is critical in cases of deep recursion to avoid stack overflow.

---

39. What are the performance implications of using for-in, for-of, and Object.keys() in large objects? When would you choose one over the other?

Answer:

for-in:

Iterates over: All **enumerable** property names (keys) in the

prototype chain.
Performance: Slowest of the three due to:

- Prototype chain traversal
- Checking each property with hasOwnProperty() if you want to skip inherited ones

for-of:

Iterates over: Iterable values, not objects by default. Works with arrays, maps, sets, etc.
For objects: You need to combine it with Object.entries(), Object.keys(), or Object.values().
Performance: Slightly slower than Object.keys() + for, due to destructuring and array creation overhead.

Object.keys() with for or forEach:

Iterates over: Only the own enumerable string keys of the object.
Performance: Faster
Use case: Preferred when you need to iterate only own properties.

Object.entries():

Similar to Object.keys(), but also returns values, offering additional flexibility. Use Object.keys() or Object.entries() when working with plain objects to avoid iterating over the prototype chain and to get more control over the keys or key-value pairs.

---

40. How do JavaScript engines (like V8) optimize code execution, and what techniques can developers use to write more engine-friendly code?

Answer:

JavaScript engines like V8 use various techniques such as Just-In-Time (JIT) compilation, hidden classes, inline caching, and escape analysis to optimize code execution.

JIT Compilation: Translates frequently executed JavaScript code into optimized machine code.

Hidden Classes: V8 optimizes objects by using hidden classes to represent their structure. Changing an object's structure dynamically (e.g., adding properties on-the-fly) can cause the engine to de-optimize the code. It's best to initialize all properties in the constructor.

Inline Caching: The engine remembers the types of objects and their properties during execution, allowing faster property access.

Escape Analysis: Determines whether an object can be safely allocated on the stack instead of the heap, improving performance. To write engine-friendly code:

Avoid dynamic object property assignment.

Avoid polymorphic functions that operate on different types of objects.

Use array operations that encourage monomorphic arrays (arrays with elements of the same type).

OUTPUT BASED QUESTIONS

41. Event Loop and Call Stack

```
console.log('Start');
setTimeout(() => {
   console.log('Timeout 1');
}, 0);
setTimeout(() => {
   console.log('Timeout 2');
}, 0);
Promise.resolve().then(() => {
   console.log('Promise 1');
});
console.log('End');
```

What will be the output?

Answer:

Start
End
Promise 1
Timeout 1
Timeout 2

Explanation:
1. console.log('Start') logs "Start".
2. setTimeout functions are added to the task queue (they will execute after the current call stack is cleared).
3. Promise.resolve() is resolved, and its then callback is added to the microtask queue, which has higher priority than the task

queue.
4. console.log('End') logs "End".
5. The microtask queue is processed next, logging "Promise 1".
6. Finally, the event loop processes the task queue, logging "Timeout 1" and "Timeout 2" in the order they were added.

---

42. Temporal Dead Zone

```
console.log(a); // What will this log?
let a = 5;
console.log(a);
```

What will be the output?

Answer:

ReferenceError: Cannot access 'a' before initialization

Explanation: The variable a is declared **using let, which is block-scoped and has a temporal dead zone** from the start of the block until the declaration is encountered. Therefore, the first console.log(a) throws a ReferenceError because a is not initialized yet.

---

43. toString()

```
const a = {};
console.log(a.toString());
```

What will be the output?

Answer:

"[object, Object]"

JavaScript objects inherit from Object.prototype.
When you create any plain object like this: const obj = {};
It automatically inherits methods from Object.prototype, including: Object.prototype.toString()

This is the default way JavaScript turns an object into a string if you don't override it.
Now, What does Object.prototype.toString() return?
Internally, Object.prototype.toString() uses a spec-defined method that returns: "[object Type]"

In this case Type is Object so it returns "[object, Object]"

---

44. Async/Await and Promise Chaining

```
async function test() {
   console.log('Start');
   await Promise.resolve();
   console.log('Middle');
   return Promise.resolve('End');
}
test().then(console.log);
```

What will be the output?

Answer:

Start
Middle
End

Explanation:
1. The test() function starts executing and logs "Start".
2. The await pauses execution until the promise resolves, allowing the event loop to run other tasks (but there are none).

3. Once the promise resolves, it continues, logging "Middle".
4. The function returns a resolved promise with the value "End", which is then logged by the outer then().

---

45. Closure and Asynchronous Behavior

```
function createFunctions() {
    let funcs = [];
    for (var i = 0; i < 3; i++) {
        funcs[i] = function() {
            console.log(i);
        };
    }
    return funcs;
}
let functions = createFunctions();
functions[0](); // What will this log?
functions[1]();// What will this log?
functions[2]();// What will this log?
```

What will be the output?

Answer:

3
3
3

Explanation: The for loop uses var, which is function-scoped. By the time the functions are invoked, i has already been incremented to 3. Each of the closures refers to the same variable i, so they all log 3.

Solution: To fix this, you can use let:

```
for (let i = 0; i < 3; i++) {
```

```
  funcs[i] = function() {
    console.log(i);
  };
}
```

This will result in logging 0, 1, and 2 respectively.

---

46. SetTimeout with Closure

```
function timer() {
  for (var i = 0; i < 3; i++) {
    setTimeout(function() {
      console.log(i);
    }, 1000);
  }
}
timer(); // What will this log after 1 second?
```

What will be the output?

Answer:

3
3
3

Explanation: Similar to the previous question,
var is **function-scoped**, so there's only one **shared i.**
After 1 second, the loop is already finished, and i === 3.
Each setTimeout callback closes over that single i, which is now 3.

Solution: Use let (block-scoped) or Use an Immediately Invoked Function Expression (IIFE) to create a new scope

```
function timer() {
  for (var i = 0; i < 3; i++) {
    (function(index) {
      setTimeout(function() {
```

```
            console.log(index);
        }, 1000);
    })(i);
  }
}
```

---

47. Chaining Promises

```
function promiseChain() {
   return Promise.resolve(1)
      .then(result => {
         console.log(result); // What will this log?
         return result + 1;
      })
      .then(result => {
         console.log(result); // What will this log?
         return Promise.resolve(result + 1);
      })
      .then(result => {
         console.log(result); // What will this log?
      });
}
promiseChain();
```

What will be the output?

Answer:

1
2
3

Explanation:
1. The first then() logs the initial resolved value 1.
2. The second then() logs 2, which is the result of the previous value incremented by 1.
3. The third then() logs 3, which is the result of resolving the promise from the second then().

48. Promise.all with Mixed Promises

```
const promise1 = Promise.resolve(1);
const promise2 = new Promise((resolve) => setTimeout(resolve, 100, 2));
const promise3 = new Promise((_, reject) => setTimeout(reject, 50,
'Error'));

Promise.all([promise1, promise2, promise3])
    .then(result => {
        console.log(result);
    })
    .catch(error => {\’’\’\
        console.log(error); // What will this log?
    });
```

What will be the output?

Answer:

Error

Explanation: Promise.all waits for all promises to resolve or for one to reject. Since promise3 rejects after 50ms, the entire Promise.all will reject and the catch block will execute, logging the error message 'Error'. The resolved values of promise1 and promise2 are ignored because of the rejection.

---

49. Async/Await with Error Handling

```
async function fetchData() {
    try {
        let response = await Promise.resolve('Data');
        throw new Error('Something went wrong!');
        return response;
    } catch (error) {
```

```
        console.log(error.message); // What will this log?
        return 'Fallback Data';
    }
}
fetchData().then(data => {
    console.log(data); // What will this log?
});
```

What will be the output?

Answer:

Something went wrong!
Fallback Data

Explanation:
1. The await expression successfully resolves to 'Data', but the following line throws an error.
2. The catch block captures this error, logging 'Something went wrong!'.
3. The catch block then returns 'Fallback Data', which is logged in the final then() method.

---

50. Variable Hoisting

You have the following code. What will be the output, and why?

```
console.log(a);
var a = 10;
console.log(b);
let b = 20;
```

Answer:

undefined

ReferenceError: Cannot access 'b' before initialization

Explanation:
Variables declared with var are hoisted and initialized with undefined at the top of the scope, so the first console.log(a) outputs undefined.
However, **variables declared with let are also hoisted, but they are in a "temporal dead zone"** until the line where they are declared is executed. Therefore, console.log(b) throws a ReferenceError.

---

51. Deep vs. Shallow Copy

You have the following object:

```
let obj1 = {
  name: 'John',
  address: {
    city: 'New York',
    zip: '10001'
  }
};

let obj2 = { ...obj1 };
obj2.address.city = 'Los Angeles';

console.log(obj1.address.city);
```

What will be the output?

Answer:

Los Angeles

Explanation:
The spread operator ({ ...obj1 }) creates a shallow copy of obj1. The address property is a nested object, and in a shallow copy, the reference to this nested object is copied, not the actual

object. Thus, modifying obj2.address.city also affects obj1.address.city.

Deep Copy Solution: To ensure a deep copy, use a method like JSON.parse(JSON.stringify(obj1)) (though this has limitations with functions and undefined values) or use a library like Lodash's cloneDeep.

---

52. Closure in Loops

```
for (var i = 0; i < 5; i++) {
  setTimeout(() => console.log(i), 1000);
}
```

What will be the output?
How can you modify this code to log 0, 1, 2, 3, 4 instead of 5 five times?

Answer:

5
5
5
5
5

Explanation:
The var keyword has function-level scope, so by the time the callback function in setTimeout is executed, the value of i has already been incremented to 5. Therefore, all callbacks print 5.

Solution:
You can use let instead of var to create block-scoped variables:

```
for (let i = 0; i < 5; i++) {
  setTimeout(() => console.log(i), 1000);
```

```
}
```

Alternatively, you can use an IIFE (Immediately Invoked Function Expression) to create a new scope:

```
for (var i = 0; i < 5; i++) {
  (function(i) {
    setTimeout(() => console.log(i), 1000);
  })(i);
}
```

---

53. Prototypes

```
const x = () => {};
console.log(x.prototype);
```

What will be the output, and why?

Answer:

Undefined

Explanation:
x is an arrow function: `const x = () => {};`
In JavaScript, **arrow functions do not have a prototype property.**
Only normal functions (declared with function or Function) have a prototype property because they can be used as constructors (with new).
Arrow functions cannot be used with new, and thus JavaScript does not assign them a .prototype.

---

54. Call, Apply, and Bind

```
const person = {
```

```
  name: 'Alice',
  greet: function(greeting) {
    console.log(`${greeting}, my name is ${this.name}`);
  }
};

const anotherPerson = {
  name: 'Bob'
};

person.greet.call(anotherPerson, 'Hello');
person.greet.bind(anotherPerson, 'Hi')();
```

What will be logged to the console, and what's the difference between call and bind?

Answer:

Hello, my name is Bob
Hi, my name is Bob

Explanation:

call immediately invokes the function with this set to anotherPerson and passes 'Hello' as an argument.

bind returns a new function with this set to anotherPerson, and 'Hi' is pre-set as an argument. When that new function is invoked, it logs the output.

---

55. Async/Await and Error Handling

```
async function fetchData() {
  try {
    let result = await fetch('https://invalidurl.com');//invalid url
    let data = await result.json();
    console.log(data);
```

```
  } catch (error) {
    console.log('Error:', error.message);
  }
}

fetchData();
```

What will be the output?

Answer:

Error: Failed to fetch

Explanation:
The fetch call fails because the URL is invalid. The try...catch block catches the error, and the error message (Failed to fetch) is logged.

Solution:
The try...catch is already handling the error. If further error handling is needed (e.g., retries), you can add logic in the catch block.

---

56. Object Freezing and Modification

```
const obj = Object.freeze({
  name: 'John',
  address: {
    city: 'New York'
  }
});

obj.name = 'Doe';
obj.address.city = 'Los Angeles';

console.log(obj);
```

What will be logged, and why? How can you truly freeze the object?

Answer:

{ name: 'John', address: { city: 'Los Angeles' } }

Explanation:

Object.freeze() makes the top-level properties immutable, but it doesn't perform a deep freeze. Therefore, you can't change name, but obj.address.city can still be modified since the nested object isn't frozen.

Solution:
To deeply freeze the object, you can use a recursive function:

```
function deepFreeze(obj) {
  Object.freeze(obj);
  Object.keys(obj).forEach(key => {
    if (typeof obj[key] === 'object' && !Object.isFrozen(obj[key])) {
      deepFreeze(obj[key]);
    }
  });
}

deepFreeze(obj);
```

---

57. Array Modification with Splice and Slice

```
let arr = [1, 2, 3, 4, 5];
let newArr = arr.splice(2, 1);

console.log(arr); // ?
console.log(newArr); // ?
```

What will be logged, and what's the difference between splice and slice?

Answer:

[1, 2, 4, 5]
[3]

Explanation:

splice(2, 1) removes 1 element from the array starting at index 2 (i.e., removes 3). It modifies the original array and returns the removed element(s).

slice, on the other hand, does not modify the original array but returns a shallow copy.

---

58. Hoisting and Function Declarations

```
console.log(foo);
foo();

var foo = function() {
  console.log('Hello');
};

function foo() {
  console.log('World');
}
```

What will be the output and why?

Answer:

[Function: foo]
TypeError: foo is not a function

Explanation:
Function declarations (function foo()) are hoisted before

var declarations. Therefore, the console.log(foo) logs the function foo().
However, the subsequent foo() call throws a TypeError because the foo variable is overwritten by var foo = function() {} which is undefined at the time of the call.

---

59. What are the key differences between a Web Worker and a Shared Worker?

Answer:

Web Worker: A Web Worker is a JavaScript execution context that runs in the background, separate from the main thread of a web application. It allows for multi-threading, enabling complex computations to be performed without blocking the user interface. Each Web Worker has its own global context and is isolated, meaning it cannot access the DOM or any variables in the main thread. It communicates with the main thread via postMessage(), which sends messages back and forth.

Shared Worker: A Shared Worker also runs in the background but can be shared among multiple browser contexts (like different tabs, windows, or iframes) originating from the same script. This allows for the sharing of state and data among different instances of the application. Shared Workers can access a broader scope, and they maintain a single instance for all clients, making them suitable for applications that require a consistent shared state, such as real-time chat applications.

---

60. What is the primary difference between for-in and for-of loops in JavaScript?

Answer:

The **for-in loop** is designed to iterate over the **enumerable**

properties of an object, including those in its prototype chain. It returns the keys (property names) of the object and is mainly used with objects. However, using for-in on arrays can lead to unexpected behavior, as it will also iterate over any inherited properties.

In contrast, the **for-of loop** iterates over the values of **iterable** objects (like arrays, strings, maps, and sets). It does not iterate over object keys and is more suitable for working with arrays. Using for-of on an array gives you the actual elements, not their indices, making it more intuitive for most scenarios. Additionally, for-of works with any iterable, providing a more general approach to looping over collections.

---

61. What is the purpose of the nullish coalescing operator (??) in JavaScript?

Answer:

The nullish coalescing operator (??) allows you to provide a default value when dealing with null or undefined. Unlike the logical OR operator (||), which treats any falsy value (like 0, NaN, or '') as a trigger for returning the right-hand side operand, the nullish coalescing operator only considers null and undefined. This distinction is crucial when you want to assign a default value but still allow legitimate falsy values to pass through. For example:

```
const value = null;
const result = value ?? 'default'; // 'default'
```

In this case, since value is null, result receives 'default'. However, if value were 0, result would be 0, which is often desired behavior.

---

62. What is the key difference between map() and forEach() in terms of return values?

Answer:

The map() method creates a new array populated with the results of calling a provided function on every element in the calling array. It transforms the original array into a new one based on the logic defined in the callback. This is useful for creating modified versions of an array **without mutating the original.**

The forEach() method, on the other hand, executes a provided function once for each array element but returns undefined. It is typically used for executing side effects, such as logging or modifying external variables, rather than transforming the data.

For example:

```
const numbers = [1, 2, 3];
const doubled = numbers.map(num => num * 2); // [2, 4, 6]
numbers.forEach(num => console.log(num)); // Logs 1, 2, 3 to console
```

---

63. What is the difference between the Callback Queue (or Task Queue) and the Microtask Queue?

Answer:

Callback Queue handles asynchronous tasks that are triggered by events such as user interactions or timers (e.g., setTimeout, setInterval). It operates on a first-in, first-out basis, meaning the tasks are executed in the order they are added to the queue.

Microtask Queue is used for tasks that are usually initiated by promises and MutationObserver callbacks. Microtasks are

prioritized over tasks in the Callback Queue. After the currently executing script (or task) completes, the event loop will process all microtasks in the Microtask Queue before moving on to the next task in the Callback Queue. This behavior allows promises to resolve quickly and ensures that any dependent microtasks execute before any new events or tasks are handled.

---

64. What does Promise.all() return, and how does it behave when passed an array of promises?

Answer:

Promise.all() takes an iterable (usually an array) of promises and returns a single promise that resolves when all of the promises in the iterable have resolved, or rejects if any of the promises reject. If all promises resolve, the resulting promise resolves with an array containing the results of each promise in the same order they were passed.

If one of the promises rejects, Promise.all() immediately rejects with the reason of the first promise that rejects, ignoring any remaining promises. This behavior allows you to handle multiple asynchronous operations and ensures that you only proceed once all operations are complete or handle any failures appropriately.

Example:

```
const promise1 = Promise.resolve(3);
const promise2 = 42; // Non-promise value is coerced into a resolved promise
const promise3 = new Promise((resolve, reject) => setTimeout(resolve, 100, 'foo'));

Promise.all([promise1, promise2, promise3]).then(values => {
  console.log(values); // [3, 42, 'foo']
});
```

65. What is "promisification," and why is it useful in modern JavaScript?

Answer:

Promisification is the process of converting functions that use callbacks for asynchronous operations into functions that return promises. This is particularly useful for working with legacy code or libraries that are not promise-based. It allows developers to leverage the power of promises and async/await, making code easier to read, write, and maintain.

By promisifying a function, you can handle success and error cases using .then() and .catch() methods or try/catch blocks with async/await, leading to cleaner code with less nested callback hell.

Example of promisification:

```
const fs = require('fs');

// Original callback-based function
function readFileCallback(filename, callback) {
  fs.readFile(filename, 'utf8', (err, data) => {
    if (err) return callback(err);
    callback(null, data);
  });
}

// Promisified version
function readFilePromise(filename) {
  return new Promise((resolve, reject) => {
    fs.readFile(filename, 'utf8', (err, data) => {
      if (err) return reject(err);
      resolve(data);
    });
  });
}
```

66. What are dynamic imports, and why are they useful in JavaScript?

Answer:

Dynamic imports allow you to load modules asynchronously at runtime using the import() function, rather than statically importing them at the top of the file. This capability enables lazy loading of modules, where you can load code only when it is actually needed, rather than at the initial load time of the application.

This can significantly improve performance, particularly for large applications or when certain features are rarely used. It also allows for conditional loading of modules based on runtime conditions, optimizing resource usage.

Example:

```
document.getElementById('loadButton').addEventListener('click',
async () => {
  const module = await import('./module.js');
  module.doSomething(); // Using the dynamically imported module
});
```

67. What is a Proxy in JavaScript, and what are its use cases?

Answer:

A Proxy is a built-in JavaScript object that allows you to create a wrapper around another object to intercept and redefine fundamental operations on that object (such as property lookup, assignment, enumeration, and function invocation). This

enables you to customize the behavior of objects at a low level.

Use cases for Proxies include:

Validation: Intercepting property assignments and validating values before they are set.
Logging: Tracking access or modifications to an object for debugging or analytics.
Creating virtual objects: Implementing objects that don't exist yet or abstracting away complex behavior behind a simpler interface.

Example:

```
const target = {};
const handler = {
  get: (obj, prop) => {
    console.log(`Getting ${prop}`);
    return prop in obj ? obj[prop] : 'Property does not exist';
  },
  set: (obj, prop, value) => {
    console.log(`Setting ${prop} to ${value}`);
    obj[prop] = value;
    return true;
  }
};
const proxy = new Proxy(target, handler);
proxy.name = 'Alice'; // Logs: Setting name to Alice
console.log(proxy.name); // Logs: Getting name,
```

Outputs: **Alice**

---

68. What is the Reflect API in JavaScript, and how does it relate to Proxies?

Answer:

The Reflect API is a built-in object in JavaScript that provides

methods for interceptable JavaScript operations. It serves as a utility for working with objects and is often used in conjunction with Proxies to enhance their functionality. The methods in the Reflect API can be seen as a way to perform operations on objects without triggering any interception or proxy handlers, ensuring that the default behavior of these operations is maintained.

For example, if you want to define a proxy handler for getting properties but still want to perform the default get operation, you can use Reflect.get() instead of directly accessing the property. This makes the code cleaner and keeps the default behavior intact, allowing for easier customization of object behavior.

Example:

```
const target = {
  name: 'Alice'
};
const handler = {
  get: (obj, prop) => {
    console.log(`Accessing property: ${prop}`);
    return Reflect.get(obj, prop); // Default behavior
  }
};
const proxy = new Proxy(target, handler);
console.log(proxy.name); // Logs: Accessing property: name, Outputs:
Alice
```

---

69. What is the purpose of Promise.race([...promises]) in JavaScript?

Answer:

Promise.race([...promises]) is a method that takes an iterable (usually an array) of promises and returns a single promise that resolves or rejects as soon as one of the promises in the iterable

resolves or rejects. This means that Promise.race() does not wait for all promises to settle; it only cares about the outcome of the first promise to settle, making it useful in scenarios where you want to proceed as soon as any of the given promises has completed.

This is particularly beneficial in situations like implementing timeouts for asynchronous operations, where you might want to proceed if one promise completes faster than another, or to react to the first response received from a set of concurrent requests.

Example:

```
const promise1 = new Promise((resolve) => setTimeout(resolve, 100,
'First'));
const promise2 = new Promise((resolve) => setTimeout(resolve, 200,
'Second'));
const promise3 = new Promise((_, reject) => setTimeout(reject, 50,
'Third'));

Promise.race([promise1, promise2, promise3])
 .then(result => {
   console.log(result); // Outputs: 'Third'
 })
 .catch(error => {
   console.error(error); // This will log if a promise rejects first});
```

---

70. How do Promise.all, Promise.allSettled, Promise.race, and Promise.any differ in JavaScript?

Answer:

These four methods are used for handling multiple promises in JavaScript, but they behave differently:

Promise.all: Resolves when all are fulfilled; rejects if any fail.

Promise.allSettled: Resolves when all are settled, regardless of their outcome.

Promise.race: Resolves/rejects as soon as one settles.

Promise.any: Resolves when at least one fulfills; rejects if all fail.

```
const p1 = Promise.resolve("A");
const p2 = Promise.reject("B");
const p3 = new Promise(res =>
  setTimeout(() => res("C (delayed)"), 100)
);

Promise.all([p1, p3]) // resolves if all succeed
  .then(console.log)  // ["A", "C (delayed)"]
  .catch(console.error);

Promise.allSettled([p1, p2]) // always resolves with status of each
  .then(console.log);        // [{ status: "fulfilled", value: "A" },
                             //  { status: "rejected", reason: "B" }]

Promise.race([p3, p1]) // first to settle (resolve or reject)
  .then(console.log);  // "A"

Promise.any([p2, p3]) // first to fulfill (ignores rejections)
  .then(console.log)  // "C (delayed)"
  .catch(console.error);
```

Method	Resolves When	Rejects When	Notes
`Promise.all`	All fulfilled	Any rejected	Fails fast
`Promise.allSettled`	All settled	Never	Always returns results for all
`Promise.race`	First settles	First settles	First win: resolve or reject
`Promise.any`	First fulfilled	All rejected	Ignores failures unless **all** fail

Chapter 3: Frontend Concepts

1. What is Local Storage in the browser? How does it differ from Session Storage?

Answer:

Local Storage is a web storage object that allows the storage of key-value pairs in the user's browser that persists even after the browser is closed. It has a larger storage capacity (typically 5-10 MB per domain) and remains available until explicitly deleted by the user or via JavaScript.
Unlike Session Storage, which is cleared when the session ends (i.e., when the browser tab or window is closed), Local Storage data persists indefinitely.

Feature	Local Storage	Session Storage
Lifetime	Persistent across sessions	Until the tab/window is closed
Scope	Shared across all same-origin tabs/windows	Unique per tab/window
Typical Use Cases	User preferences, caching data between visits	Transient state (e.g. form data), per-tab workflows

---

2. What data types can be stored in Local Storage?

Answer:

Local Storage (and Session Storage) store only string key–value

pairs. Any value you save is implicitly converted to a string.

Other data types like objects or arrays must be serialized using JSON.stringify() before storage and deserialized with JSON.parse() when retrieved.

---

3. How do you store and retrieve data in Local Storage?

Answer:

```
// Storing data
localStorage.setItem('key', 'value');
localStorage.setItem('user', JSON.stringify({ name: 'John', age: 30 }));

// Retrieving data
const value = localStorage.getItem('key');
const user = JSON.parse(localStorage.getItem('user'));
```

---

4. What are the potential security risks of using Local Storage?

Answer:

Local Storage is not a secure storage mechanism. It is **vulnerable to cross-site scripting (XSS) attacks**, where an attacker can access and manipulate stored data.
It should never be used to store sensitive information like passwords or tokens.

---

5. Can Local Storage be shared across domains?

Answer:

No, Local Storage is scoped to the origin (protocol + domain + port). It cannot be shared across different domains or subdomains due to the same-origin policy.

---

6. What is Session Storage and how is it used in modern web applications?

Answer:

Session Storage is a web storage object similar to Local Storage but its data is only available for the duration of the page session (i.e., as long as the browser tab or window is open).
Once the session ends, the stored data is automatically deleted. It's commonly used for storing temporary data like UI states or form inputs.

---

7. How do you store and retrieve data in Session Storage?

Answer:

```
// Storing data
sessionStorage.setItem('key', 'value');
sessionStorage.setItem('cart', JSON.stringify([1, 2, 3]));

// Retrieving data
const value = sessionStorage.getItem('key');
const cart = JSON.parse(sessionStorage.getItem('cart'));
```

---

8. Can Session Storage be accessed by other tabs or windows?

Answer:

No, Session Storage is unique to the specific tab or window in which it is created. Even tabs with the same URL in the same browser will have different Session Storage objects.

---

9. What happens to Session Storage data when the page is reloaded?

Answer:

Session Storage persists across page reloads, but the data will be cleared when the browser tab or window is closed.

---

10. What are cookies, and how do they differ from Local and Session Storage?

Answer:

Cookies are small pieces of data sent by a web server to the user's browser, stored either temporarily (session cookies) or persistently. **Unlike Local and Session Storage, cookies can be sent with every HTTP request to the server**, making them useful for **server-side session management** but less ideal for large amounts of data due to their size limit (4KB).

Feature	Cookies	Session Storage	Local Storage
Storage Capacity	~4 KB per cookie (varies by browser)	~5-10 MB (varies by browser)	~5-10 MB (varies by browser)
Expiration	Set by developer (can expire or persist until manually cleared)	Expires when browser tab is closed	Persists until explicitly cleared
Scope	Accessible across all tabs/windows of the same domain (based on path/domain)	Limited to the specific browser tab	Accessible across all tabs/windows of the same domain
Data Type	Key-value pairs (strings)	Key-value pairs (strings)	Key-value pairs (strings)
Accessibility	Accessible by client (JavaScript) and server (via HTTP headers)	Client-side only (JavaScript)	Client-side only (JavaScript)
HTTP Transmission	Automatically sent with every HTTP request to the domain	Not sent with HTTP requests	Not sent with HTTP requests
Use Case	User authentication, tracking, preferences (e.g., session IDs, analytics)	Temporary data for a single session (e.g., form data)	Persistent data (e.g., app settings, offline data)
Security	Can be secured with `HttpOnly`, `Secure`, and `SameSite` attributes	No server access, inherently safer for client-side data	No server access, inherently safer for client-side data
Performance Impact	Adds overhead to HTTP requests due to automatic transmission	No network overhead	No network overhead
API	`document.cookie` (complex to parse)	`window.sessionStorage` (simple key-value API)	`window.localStorage` (simple key-value API)

When to Use Which:

Cookies

Use for data that the server needs on each request (auth tokens, session IDs), or when you need flags like HttpOnly/Secure for security.

Local Storage

Use for larger, client-only data you want to persist across browser sessions (e.g. UI preferences, offline caches).

Session Storage

Use for state that should reset when the tab closes (e.g. in-tab wizards, one-time tokens).

11. How can you create, read, and delete cookies in JavaScript?

Answer:

```
// Create a cookie
document.cookie = "username=John; expires=Fri, 31 Dec 2024 12:00:00 UTC; path=/";

// Read all cookies
const allCookies = document.cookie;

// Delete a cookie
document.cookie = "username=John; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/";
```

12. What are HttpOnly and Secure cookie attributes?

Answer:

The HttpOnly and Secure attributes are security-related flags you can set on cookies to help protect sensitive data and prevent common web attacks.

HttpOnly Attribute:

Prevents the cookie from being accessed via JavaScript (document.cookie).

- Helps prevent Cross-Site Scripting (XSS) attacks from stealing cookies.
- Keeps sensitive data like session tokens safe from client-side scripts.

Example:

```
Set-Cookie: sessionId=abc123; HttpOnly
```

JavaScript cannot access this cookie:

```
console.log(document.cookie); // sessionId won't show if it's HttpOnly
```

Secure Attribute

Ensures the cookie is only sent over HTTPS connections.

- Prevents cookie interception over unsecured (HTTP) networks.
- Protects data from man-in-the-middle (MITM) attacks.

Example:

```
Set-Cookie: sessionId=abc123; Secure
```

This cookie won't be sent if the site is accessed over HTTP.

Combine Them for Best Security

Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict

HttpOnly: Protects against XSS.
Secure: Protects against MITM.
SameSite: Defends against CSRF (optional but recommended).

Attribute	Purpose	Protects Against	Access via JS	Sent Over HTTP
`HttpOnly`	Hides cookie from JavaScript	XSS attacks	❌ No	✅ Yes
`Secure`	Sends only over HTTPS	MITM attacks	✅ Yes (if not HttpOnly)	❌ No

---

13. What is IndexedDB, and why is it used in web development?

Answer:

IndexedDB is a low-level, NoSQL database that allows the storage of large amounts of structured data in the browser.

-**A NoSQL database**: object store-based, not relational.
-**Allows storage of complex, structured data** (unlike localStorage, which stores only strings).
-**Asynchronous and event-driven**: uses promises or event listeners for operations.

Why Use IndexedDB?

Benefit	Description
Large storage	Can store **hundreds of MBs** (or even more, depending on browser).
Structured data	Stores JS objects, arrays, blobs, etc. with automatic serialization.
Indexing	Supports indexes on object properties for **fast querying**.
Works offline	Great for **PWAs** and offline web apps.
No server round-trips	Everything runs in the browser — no need for network access.

---

14. How is data stored and retrieved from IndexedDB?

Answer:

```
// Opening or creating a database
const request = indexedDB.open('MyDatabase', 1);

request.onsuccess = function(event) {
  const db = event.target.result;

  // Create a transaction and store data
  const transaction = db.transaction('users', 'readwrite');
  const store = transaction.objectStore('users');
  store.put({ id: 1, name: 'John', age: 30 });
```

```
  // Retrieve data
  store.get(1).onsuccess = function(event) {
    console.log(event.target.result);
  };
};
```

---

15. What are the advantages of IndexedDB over Local Storage?

Answer:

Feature	IndexedDB Advantage	Local Storage Limitation
Storage Capacity	Much larger (often 100s of MB to GBs, depending on browser and disk space)	Limited to ~5-10 MB (varies by browser)
Data Structure	Supports complex data (objects, arrays, binary data) with key-value and indexed queries	Only stores simple key-value pairs (strings)
Querying	Allows advanced querying (e.g., indexes, cursors, ranges) for efficient data retrieval	No querying; requires manual parsing and iteration
Performance	Optimized for large datasets with asynchronous operations	Slower for large data due to synchronous operations
Data Types	Natively supports JavaScript objects, blobs, and files	Requires serialization (e.g., JSON) to store non-string data
Scalability	Scales well for large, structured datasets (e.g., offline apps, PWAs)	Not suited for large or complex datasets
Transaction Support	Provides transactions for data integrity and atomic operations	No transaction support; manual error handling

Real-World Scenarios Where IndexedDB Wins :

Offline Web Apps: Store user data like notes, drafts, emails, etc.
Data-Driven UIs: Cache complex API responses and enable instant interactions.
Games: Store player progress, settings, or assets.
Media Apps: Store large image/audio/video blobs locally.

When *Not* to Use IndexedDB

If you're storing **a few small, simple strings** like theme preferences or auth flags, localStorage might still be simpler and sufficient.

---

16. What are some limitations of IndexedDB?

Answer:

IndexedDB is less user-friendly than Local Storage due to its complex API and asynchronous nature.
It’s not ideal for small, simple datasets or scenarios requiring synchronous access.
Developers must handle security, browser quirks, and potential data loss manually, which adds complexity to implementation.
Additionally, the browser’s storage quota can vary depending on the user’s device and available disk space.

---

17. What is HTTP caching and why is it important for performance?

Answer:

HTTP caching is a performance technique where responses from a server (like HTML, CSS, JavaScript, images, etc.) are **stored temporarily** — either in the **browser**, a **proxy**, or a **CDN** — so that future requests for the same resources can be served **faster**, without re-downloading them from the origin server.

When the browser makes a request, it checks:

- Do I already have this in cache?
- Is it still valid? (based on headers like Cache-Control, Expires, ETag, etc.)

If yes, it skips a network call and uses the cached version.

HTTP caching:

- Boosts performance by reducing latency and load times
- Reduces server strain and bandwidth costs
- Is controlled via HTTP headers
- Is essential for modern web speed and scalability

Common Caching Headers:

Header	Purpose
`Cache-Control`	Defines caching rules (`max-age` , `no-cache` , `public` , `private`)
`Expires`	Sets an explicit expiration date (less preferred than `Cache-Control`)
`ETag`	A unique identifier for a resource — used for revalidation
`Last-Modified`	Indicates when the resource was last changed

---

18. What is the difference between Cache-Control and Expires headers?

Answer:

The Cache-Control and Expires headers both control how and for how long a browser (or proxy) caches a response, but they differ in flexibility, accuracy, and usage.

Cache-Control is a more modern header that allows precise control over caching policies (e.g., max-age, no-cache, private).

Expires sets a specific date and time when the cached resource will become stale. It is less flexible and is often replaced by Cache-Control.

✅ `Cache-Control` **(Modern)**

```
Cache-Control: public, max-age=3600
```

➡️ Tells the browser to cache this resource for **1 hour (3600 seconds)** after it's received.

🕒 `Expires` **(Legacy)**

```
Expires: Thu, 20 Apr 2025 10:00:00 GMT
```

➡️ Tells the browser the **exact date/time** the resource becomes stale.

---

19. Explain the difference between no-cache and no-store in Cache-Control.

Answer:

no-cache:
Allows the response to be cached
But requires revalidation with the server before each use

Use case: When data can be reused if it's still fresh — like an API response that may change often.

```
Cache-Control: no-cache
```

Browser stores it, but will ask the server if it's still valid before using it.

no-store:
Prevents storing the response anywhere (not in browser, not in proxy)
Nothing is cached at all

Use case: For sensitive data, like login responses, banking details, or

tokens.

```
Cache-Control: no-store
```

Browser will not keep any copy — must fetch fresh every time.

---

20. What are workers and what are different types of types of workers? (THIS QUESTION WAS ASKED IN NASDAQ)

Answer:

In JavaScript, especially in browser environments, Workers are a **way to run scripts in background threads**, allowing you to perform expensive or blocking tasks without freezing the main UI thread.

Workers provide a concurrent execution model in JavaScript. They:

- Run in separate threads
- Can communicate with the main thread via messages
- Do not have access to the DOM
- Are great for heavy computations, file processing, data parsing, etc.

Types of Workers:

Worker Type	Description
Web Workers	Standard background workers; run a script in a separate thread
Shared Workers	Can be shared between multiple scripts (even across tabs/windows of same origin)
Service Workers	Act as a proxy between web app and network; used for caching & offline support

---

21. What is a Service Worker, and how is it related to caching?

Answer:

A Service Worker is a special type of web worker that acts as a proxy between your web app and the network. It runs in the background, intercepts network requests, and enables powerful features like offline support, smart caching, and background sync — all without blocking the main thread. It is:

- A JavaScript file that the browser runs in the background
- Works independently of the web page (can work even when the page is closed)
- Can intercept and handle fetch events — ideal for custom caching strategies

Service Workers give you full control over what gets cached, when, and how:
- Cache dynamic API responses or static assets
- Serve cached assets when offline
- Build Progressive Web Apps (PWAs) with full offline capability

Service Workers = Background workers + Smart caching

Let you intercept requests and serve from cache or custom logic
Critical for building fast, resilient offline-capable web apps

App	Uses Service Worker?	Purpose
Google Maps	✅ Yes	Cache map tiles & scripts, support limited offline use
Twitter Lite	✅ Yes	App shell caching, offline tweets, faster reloading
Spotify PWA	✅ Yes	Enable playback & offline playlist features in-browser
Starbucks PWA	✅ Yes	Offline ordering, background sync
Flipkart Lite	✅ Yes	Fast loading + offline browsing of products

---

22. How does the browser rendering pipeline

work from receiving an HTML file to displaying it on the screen?

Answer:

The browser rendering pipeline is the step-by-step process a browser follows to turn an HTML document into pixels on the screen :

1. HTML Request & Download
    - Browser sends a request (HTTP/HTTPS)
    - Receives the HTML file from the server

2. HTML Parsing & DOM Construction
    - HTML is parsed top to bottom
    - Builds the DOM (Document Object Model)
    - Encountering a <script> can block parsing unless it's async or defer

3. CSS Parsing & CSSOM Construction
    - CSS files and inline styles are parsed
    - Builds the CSSOM (CSS Object Model) — a tree of styling rules

4. Render Tree Construction
    - Browser combines DOM + CSSOM → builds the Render Tree
    - Each visible node (like , <p>) is mapped with computed styles

5. Layout (Reflow)
    - Calculates position and size of each element
    - Based on styles like display, position, width, height, etc.

6. Painting
    - Each node in the render tree is converted into visual pixels
    - Backgrounds, text, borders, shadows, etc.

7. Compositing
    - Browser layers are combined in the correct order
    - Uses GPU if hardware acceleration is enabled

8. Rasterization & Display
    - Final visual representation is rasterized (converted to pixels)
    - Displayed on screen via the GPU
    -

```
HTML --> DOM
CSS  --> CSSOM
DOM + CSSOM --> Render Tree
Render Tree --> Layout --> Paint --> Composite --> Screen
```

This whole flow is called the **Critical Rendering Path**

---

23. How does the CSS box model influence layout decisions, and what are its components?

Answer:

The CSS Box Model is the foundation of how browsers calculate the size and position of elements on a page.

Components of the Box Model:

Part	Description
Content	Actual content (text, images, etc.)
Padding	Space between content and border
Border	Line around the padding & content
Margin	Space between this box and other elements

24. What is a stacking context, and how can its improper handling affect UI performance?

Answer:

A stacking context in CSS is like a **3D layer system** that determines how elements stack or overlap on the Z-axis (front-to-back).

A stacking context is automatically created when:
- The root element (<html>)
- An element with:
  - position other than static + z-index other than auto
  - opacity less than 1 (opacity: 0.99)
  - transform, filter, perspective, etc.
  - isolation: isolate
  - will-change with z-index, transform, etc.

```
.modal {
  position: relative;
  z-index: 10; /* creates new stacking context */
}
```

<u>How It Works:</u>

Elements in the same stacking context are ordered by z-index. Elements in different stacking contexts don't affect each other's z-

index values.

NPM

25. What is the difference between dependencies, devDependencies, and peerDependencies in npm?

Answer:

Dependencies: These are the packages required for your application to run in production. They are essential for the core functionality of the app.

DevDependencies: These packages are only necessary during the development process (e.g., testing, building). They are not included in the production build.

PeerDependencies: These specify packages that must be installed alongside your package, but are not automatically installed. This is commonly used for libraries that need a specific version of another library to work correctly.

```
"dependencies": {
  "react": "^18.2.0"
},
"devDependencies": {
  "webpack": "^5.90.0"
},
"peerDependencies": {
  "react": "^18.0.0"
}
```

Summary:

Dependencies are for production use, devDependencies are for development use, and peerDependencies are for compatibility with

other packages.

---

26. What is the purpose of the .npmrc file, and what kind of configurations can be specified in it?

Answer:

The .npmrc file is used **to configure npm's behavior**. It can include settings such as registry URLs, authentication tokens, and proxy settings.

The configurations can be set globally (in the user's home directory) or locally (within a specific project). Common configurations include setting a custom registry, defining package scopes, and setting the default package manager.

---

27. How can you create a package that supports both CommonJS and ES modules?

Answer:
To create a package that supports both CommonJS and ES modules, you can do the following:

Use the "type": "module" field in your package.json to indicate that the package uses ES modules.

For compatibility, create a separate entry point for CommonJS by using the .cjs file extension for those files.

Additionally, provide a module field in package.json pointing to your ES module entry point and a main field pointing to your CommonJS entry point.

Step	Description
✅ Write in ESM	Use `import/export` in your source code (e.g., `src/index.js`)
⚙️ `package.json`	Set both `main` (CJS) and `module` (ESM) fields, and configure `exports`
🔧 Build Tools	Use **Rollup** or **Babel** to transpile to both formats (`.js` for ESM, `.cjs` for CJS)
🧪 Compatibility	Consumers can use `require()` or `import` based on their environment

---

28. Explain the concept of "package-lock.json" and its significance.

Answer:

The package-lock.json file is automatically generated when running npm install. It records the exact version of every installed package and its dependencies, ensuring a consistent installation across different environments. This file helps to avoid the "it works on my machine" problem by locking the dependency tree, allowing for reproducible builds. It also speeds up installations because npm can skip the resolution step.

---

29. What are the differences between npm install --save, npm install --save-dev, and npm install --global?

Answer:

npm install --save: This installs a package and adds it to the dependencies section of package.json, indicating that it is required for the production environment.

npm install --save-dev: This installs a package and adds it to the devDependencies section, indicating it is only needed during development (e.g., testing or build tools).

npm install --global: This installs a package globally, making it available system-wide. This is often used for command-line tools.

---

30. What is the difference between caret (^) and tilde (~) in versioning in npm? (THIS QUESTION WAS ASKED IN NASDAQ)

Answer:

The caret (^) and tilde (~) are both used to define version ranges in package.json, but they behave differently:

Caret (^): Allows updates to the most recent minor version. For example, if the version is specified as ^1.2.3, it can update to any version less than 2.0.0 (e.g., 1.2.4, 1.3.0).

```
"dependencies": {
  "lodash": "~4.17.21" // Installs 4.17.x, not 4.18.0
}
```

Tilde (~): Allows updates to the most recent patch version. For example, if the version is specified as ~1.2.3, it can update to any version less than 1.3.0 (e.g., 1.2.4, but not 1.3.0).

```
"dependencies": {
  "axios": "^1.3.2" // Installs 1.x.x, but not 2.0.0
}
```

Summary:

Symbol	Meaning	Installs
~	Patch updates	Only **bug fixes** (same minor)
^	Minor + Patch updates	**New features** + **bug fixes** (same major)

CSS

31. How does CSS opacity work, and what are its implications on child elements?

Answer:

The opacity property in CSS controls the transparency level of an element, including all of its content (text, background, borders, and child elements).

```
.element {
  opacity: 0.5; /* 50% transparent */
}
```

Range: 0 (fully transparent) → 1 (fully opaque)
It applies to the entire element, including all children.

Implications on Child Elements
When you apply opacity to a parent:
All child elements become equally transparent, even if you try to style them separately.

```
<div class="parent" style="opacity: 0.5;">
  <p>This text will also be 50% transparent</p>
</div>
```

This makes it impossible to make children fully opaque unless you change your approach.

---

32. What are the key differences between Flexbox and Grid layout in CSS, and when would you choose one over the other?

Answer:

Flexbox and Grid are both powerful layout models in CSS, but they serve different purposes:

1. Flexbox:
One-dimensional layout system (either in a row or a column).
Ideal for distributing space along a single axis and aligning items within a container.
Provides properties like justify-content, align-items, and flex-grow to control layout dynamics.

2. Grid:
Two-dimensional layout system (rows and columns).
Allows for complex layouts that can involve both rows and columns simultaneously.
Offers capabilities like grid-template-areas, grid-column, and grid-row, making it easier to create intricate designs.

Aspect	Flexbox	CSS Grid
Purpose	One-dimensional layout (either row or column).	Two-dimensional layout (rows and columns simultaneously).
Layout Control	Primarily for arranging items in a single direction (horizontal or vertical).	Designed for complex layouts with precise control over both rows and columns.
Item Placement	Items are placed along a main axis, with limited control over the cross axis.	Items can be placed anywhere on a grid using explicit or implicit placement.
Alignment	Offers flexible alignment with `justify-content`, `align-items`, etc.	Provides alignment for both axes (`justify-content`, `align-content`, etc.).
Content-Driven	Content size heavily influences layout (e.g., flex items shrink/grow).	Grid is more structure-driven, with defined tracks (rows/columns).
Gap Control	Uses `gap` for spacing between flex items (modern browsers).	Uses `gap` for row and column gaps, with separate `row-gap` and `column-gap`.
Item Overlap	Limited support for overlapping (requires workarounds).	Native support for overlapping items using grid areas or `z-index`.
Grid Tracks	No concept of tracks; items flow based on flex properties.	Explicitly defined rows and columns with `grid-template-rows/columns`.

Use Flexbox when:

- You're aligning items in a single row or column.
- You need to distribute space dynamically (e.g., space-between, center).
- You're building navigation bars, toolbars, buttons, or simple forms.
- You want elements to wrap automatically across lines.

Example: A horizontal navbar, or a vertical list of items that need flexible spacing.

Use Grid when:

- You need a two-dimensional layout (rows and columns).
- You're building complex page layouts or card grids.
- You want precise placement of elements, even if they skip spaces.
- You need items to span across multiple rows or columns.

Example: A dashboard layout with a sidebar, header, content area, and widgets.

---

33. What does flex: 1 mean in a Flexbox layout?

(THIS QUESTION WAS ASKED IN JPMC)

Answer:

The flex: 1 shorthand property in Flexbox is a combination of three properties: flex-grow, flex-shrink, and flex-basis. It essentially translates to **flex: 1 1 0%.**

```
flex: 1; /* === flex-grow: 1; flex-shrink: 1; flex-basis: 0% */
```

Here's what each component means:

flex-grow (1): This allows the flex item to grow and fill the available space in the flex container. If multiple items have flex-grow set to 1, they will share the available space equally.

flex-shrink (1): This allows the flex item to shrink if necessary,

meaning it can reduce its size when the container is too small to fit all items.

flex-basis (0%): This sets the initial size of the flex item to 0, meaning it will start with no inherent size and will then grow based on the available space and the flex-grow value.

In summary, flex: 1 makes the flex item flexible, allowing it to grow and shrink to fit the available space in the container while sharing it with other flex items.

---

34. How can you create a circle using a in CSS? (THIS QUESTION WAS ASKED IN LTIMINDTREE)

Answer:

To create a circle using a , you need to set equal width and height and then apply a border-radius of 50%. Here's an example:

```
<div class="circle"></div>

.circle {
   width: 100px; /* Set a width */
   height: 100px; /* Set an equal height */
   background-color: blue; /* Set a background color */
   border-radius: 50%; /* This makes it a circle */
}
```

In this example, the border-radius property is the key to transforming the square into a circle. The width and height can be adjusted to create circles of different sizes.

---

35. What are pseudo-elements in CSS, and how are they different from pseudo-classes?

Answer:

Pseudo-elements and pseudo-classes are both used to style specific parts of an element, but they serve different purposes:
Pseudo-elements let you style specific **parts of an element's content** or **insert virtual elements** into the DOM that don't exist in the HTML.

```
selector::pseudo-element {
  /* styles */
}
```

Pseudo-element	What it does
`::before`	Inserts content **before** the element's actual content
`::after`	Inserts content **after** the element's actual content
`::first-line`	Targets the **first line** of text in a block element
`::first-letter`	Styles the **first letter** of a block of text
`::selection`	Styles text when it is **selected** by the user

Pseudo-classes target elements **based on their state** or position in the DOM.

```
selector:pseudo-class {
  /* styles */
}
```

Pseudo-class	What it does
`:hover`	When the mouse is over an element
`:active`	When the element is being clicked
`:focus`	When the element is focused (e.g. input field)
`:nth-child(n)`	Selects the nth child in a parent
`:not(selector)`	Selects every element **except** the one passed in

In summary, **pseudo-classes deal with the state of elements,** while **pseudo-elements focus on styling specific portions of the element**'s content or structure.

CORS

36. What is CORS?

Answer:

A CORS stands for **Cross-Origin Resource Sharing**. It's a security feature implemented by web browsers that controls how web pages from one origin (domain) can request resources (like APIs, images, fonts, etc.) from a different origin.

Why CORS Exists
Modern browsers block cross-origin HTTP requests by default due to same-origin policy, which prevents potentially malicious scripts on one site from accessing sensitive data on another.
CORS is a way for the server to say, "Yes, it's okay for that other site to access this resource."

How It Works
When a browser makes a request to a different origin, it sends an HTTP request with special headers, like:

```
Origin: https://example.com
```

If the server allows the request, it responds with headers like:

```
Access-Control-Allow-Origin: https://example.com
```

Or to allow all origins (not recommended for sensitive data):

```
Access-Control-Allow-Origin: *
```

Example Use Case
Let's say your frontend app is hosted at **https://myfrontend.com** and your backend API is at **https://myapi.com.** If the frontend tries to fetch data from the backend, the backend must include the appropriate CORS headers to allow that request.

Preflight Requests
For some requests (like those with custom headers or non-simple methods like PUT, DELETE), the browser sends an OPTIONS request first — called a preflight request — to check if the actual request is safe.

---

37. What is a preflight request in CORS, and why is it necessary?

Answer:

A preflight request is a CORS (Cross-Origin Resource Sharing) mechanism used by browsers to determine if it's safe to send a non-simple cross-origin HTTP request.

It's an HTTP OPTIONS request that the browser automatically sends to the server before the actual request.

The preflight request asks:
"Hey server, I'd like to send this request with some specific methods/headers. Are you okay with that?"

Why Is It Necessary?
To protect users from potentially malicious cross-origin requests that could:

- Use unexpected HTTP methods (like PUT, DELETE)
- Send custom headers
- Include non-standard content types
- Bypass security policies or exploit APIs unknowingly

The server can allow or reject the request based on its rules, before any sensitive data is sent.

When Does a Preflight Happen?
When the actual request is not a "simple request."
A simple request:

- Uses GET, POST, or HEAD
- Content-Type is application/x-www-form-urlencoded, multipart/form-data, or text/plain
- Doesn't use custom headers like Authorization, X-Custom-Header, etc.

Any deviation → preflight is triggered.

Example:

```
OPTIONS /api/data HTTP/1.1
Origin: https://example.com
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: X-Custom-Header
```

---

38. What's the difference between CORS errors and same-origin policy violations?

Answer:

Same-Origin Policy (SOP)
Built-in browser rule: Only allow requests to the same origin (same domain, protocol, and port).
Blocks JS from accessing data from another origin by default.
Violations happen when you try to access or interact with cross-origin data without permission.

CORS (Cross-Origin Resource Sharing)
A way for the server to opt-in and allow cross-origin access.
Uses headers like Access-Control-Allow-Origin to tell the browser,

"This is okay."
If these headers are missing or wrong, the browser shows a CORS error.

Feature	SOP Violation	CORS Error
Enforced by	Browser	Browser (based on server headers)
Request sent?	Often **blocked before sending**	Sent, but **response is blocked**
Fixable?	❌ No	✅ Yes, by adding CORS headers

---

39. How to resolve CORS issue?

Answer:

1. <u>On the Backend (most common fix):</u>
Add the appropriate Access-Control-Allow-Origin header in the response.
In Node.js (Express):

```
const cors = require('cors');
app.use(cors()); // Allows all origins by default
```

Or for specific domain:

```
app.use(cors({ origin: 'https://domainA.com' }));
```

2. <u>Handle Preflight Requests:</u>
For PUT, DELETE, or custom headers, the browser sends an OPTIONS request first.
Ensure the backend correctly handles these preflight OPTIONS requests and responds with:

- Access-Control-Allow-Methods
- Access-Control-Allow-Headers
- Access-Control-Allow-Origin

3. <u>Frontend workaround (dev-only):</u>
Use a proxy in development (e.g., in React, set "proxy": "http://localhost:5000" in package.json) to avoid CORS issues.
NOTE: Not a production solution, just for dev ease.

WEBPACK

40. Explain the concepts of minification, obfuscation, and tree shaking in the context of Webpack.

Answer: (THIS QUESTION WAS ASKED IN NASDAQ)

In the context of Webpack, these concepts are essential for optimizing JavaScript applications:

1. Minification: Minification is the process of removing all unnecessary characters from code (like whitespace, comments, and line breaks) without changing its functionality, to reduce the size of JavaScript, CSS, or HTML files.
Webpack uses plugins like TerserPlugin (enabled by default in production mode) to minify JS.

Example:

```
// Original
function greet(name) {
  console.log("Hello, " + name);
}

// Minified
function greet(n){console.log("Hello, "+n);}
```

2. Obfuscation: Obfuscation goes a step beyond minification by intentionally making code difficult to read or reverse-engineer. It changes variable names, function names, and structure in a way that preserves functionality but hides logic.
Webpack doesn't obfuscate by default.
You can integrate tools like javascript-obfuscator via plugins if

needed.

Example:

```
// Original
function getUser() {
  return { name: "Alice" };
}

// Obfuscated
function _0x12f4(){return{name:'\x41\x6c\x69\x63\x65'};}
```

3. Tree Shaking: Tree shaking is the process of removing unused (dead) code from the final bundle, particularly unused exports from ES6 modules.
In Webpack works best with ES Modules (ESM) and sideEffects: false in package.json.
Webpack analyzes which exports are actually used and excludes the rest.
Helps reduce bundle size significantly.

```
// utils.js
export function used() { /* ... */ }
export function unused() { /* ... */ }

// main.js
import { used } from './utils';
```

unused() won't be included in the final bundle if tree shaking is active.

Concept	Purpose	Webpack Support
Minification	Reduce file size	✅ Yes (TerserPlugin)
Obfuscation	Make code harder to reverse-engineer	🔌 Optional via plugin
Tree Shaking	Remove unused code	✅ Yes (ES6 + `sideEffects`)

==========

4 YEARS EXPERIENCED MERN STACK DEVELOPER REAL INTERVIEW REPORT